DEVOPS BIBLE

VERIQTA
ENGINEERING EXCELLENCE

THE COMPLETE DEVOPS HANDBOOK FOR ENGINEERS

# DEVOPS BIBLE

PRODUCTION REAL-WORLD FOCUS

## FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

DEVELOP → GIT & COLLABORATION → CI/CD PIPELINE → CONTAINER & KUBERNETES → CLOUD INFRASTRUCTURE → MONITORING & OBSERVABILITY

**COVERING**
- Linux
- Networking
- Cloud
- Terraform
- Docker
- Kubernetes
- CI/CD
- GitOps
- Observability
- DevSecOps
- SRE & Reliability
- Incident Mgmt
- Best Practices




**BUILT FOR**
- Developers
- DevOps Engineers
- SREs
- Platform Engineers
- Cloud Engineers
- Tech Leads
- Engineering Teams

**REAL-WORLD FOCUS**

SECURITY BY DESIGN | RELIABILITY AT SCALE | AUTOMATION EVERYWHERE | COLLABORATION & CULTURE

LEARN. BUILD. DEPLOY. OPERATE. REPEAT.

USERS → DNS → LOAD BALANCER → KUBERNETES CLUSTER → DATABASE → STORAGE → MONITORING

**VERIQTA SOCIAL**

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

PAGE 1

# VERIQTA
ENGINEERING EXCELLENCE

**DEVOPS BIBLE**
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 1. DEVOPS ENGINEERING IN THE REAL WORLD

**Build better. Deploy faster. Operate smarter.**

## WHAT DEVOPS ACTUALLY MEANS

- DevOps is a mindset, culture, and set of practices that brings development and operations together.
- It focuses on delivering value to users faster, with higher quality and reliability.
- DevOps removes silos, improves collaboration, and automates everything from code to production.
- The goal is to build, test, release, monitor, and improve continuously.

## DEVELOPMENT AND OPERATIONS RESPONSIBILITIES

### DEVELOPMENT (DEV)

- Write and maintain code
- Build and test applications
- Commit and manage source code
- Create automated tests
- Ensure code quality
- Package and version artifacts
- Work closely with operations

### OPERATIONS (OPS)

- Provision and manage infrastructure
- Deploy and run applications
- Monitor systems and applications
- Ensure availability and performance
- Manage security and access
- Handle incidents and outages
- Continuously optimize systems

## DEVOPS LIFECYCLE

| PLAN | CODE | BUILD | TEST | RELEASE | DEPLOY | OPERATE | IMPROVE |
|---|---|---|---|---|---|---|---|
| Plan features, tasks and requirements | Write code and commit changes | Compile code and build artifacts | Run automated tests and scans | Package and release to artifact repository | Deploy to staging or production | Monitor, maintain and support | Collect feedback and improve |

CONTINUOUS FEEDBACK AND IMPROVEMENT

## PEOPLE, PROCESS, AND TECHNOLOGY

**PEOPLE**
Collaboration, shared ownership, communication, and trust.

**PROCESS**
Agile, Lean, CI/CD, automation, monitoring, and continuous improvement.

**TECHNOLOGY**
Tools and platforms that automate, integrate, and provide visibility across the lifecycle.

**DEVOPS = PEOPLE + PROCESS + TECHNOLOGY**

## DEVOPS vs SRE vs PLATFORM ENGINEERING

| | |
|---|---|
| DEVOPS | Culture and practices that unite development and operations to deliver software faster and more reliably. |
| SRE | Applies engineering principles to operations. Focuses on reliability, SLIs, SLOs, error budgets, and reducing toil. |
| PLATFORM ENGINEERING | Builds internal developer platforms that provide self-service tools, scaffolding, and golden paths to speed up delivery. |

**DEVOPS IS CULTURE. SRE IS RELIABILITY. PLATFORM ENGINEERING IS ENABLING SCALE.**

## PRODUCTION ENGINEERING MINDSET

- Own the full lifecycle from code to production.
- Design for reliability, scalability, and security.
- Automate to reduce manual work and errors.
- Monitor everything that matters.
- Measure, learn, and improve continuously.
- Think in systems, not in isolated components.
- Prepare for failure; build for recovery.
- Focus on customer value and user experience.
- Take responsibility and act with urgency.
- Document, standardize, and share knowledge.

**THINK LIKE AN OWNER.
BUILD LIKE A PROFESSIONAL.
OPERATE LIKE A CHAMPION.**

## HOW MODERN ENGINEERING TEAMS OPERATE

- Cross-functional teams with shared goals
- Collaborate early and continuously
- Automate everything that can be automated
- Use cloud-native and scalable platforms
- Build security from the start
- Measure results and iterate fast

PAGE 2

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS



# 2. LINUX FOR DEVOPS ENGINEERS

**Understand. Operate. Automate. Scale.**

## 1 LINUX ARCHITECTURE

- USER APPLICATIONS
- SYSTEM LIBRARIES
- KERNEL
- CPU | RAM | DISK | NET

- Kernel manages hardware and resources
- Everything is a file
- Processes, memory, files, devices, and networking are managed by the kernel
- Shell is the user interface

## 2 FILESYSTEM HIERARCHY

| | |
|---|---|
| / | Root directory |
| /bin | Essential binaries |
| /etc | System configuration |
| /home | User home directories |
| /var | Variable data |
| /usr | User programs & data |
| /opt | Optional applications |
| /tmp | Temporary files |
| /dev | Device files |
| /proc | Process information |
| /sys | System information |
| /boot | Boot loader files |
| /lib | Shared libraries |
| /mnt | Mount point |
| /srv | Service data |

## 3 USERS AND GROUPS

- Every file has an owner (user) and group
- Create users: useradd
- Modify user: usermod
- Delete user: userdel
- Switch user: su - <user>
- View groups: groups
- Add user to group: usermod -aG
- Principle of least privilege

## 4 PERMISSIONS

```
- rwx r-x r--
Owner Group Others
  7     5     4
```

- r = Read (4)
- w = Write (2)
- x = Execute (1)
- View permissions: ls -l
- Change mode: chmod
- Change owner: chown
- Change group: chgrp
- Use least privilege always

## 5 PROCESSES

- A process is a running instance of a program
- View processes: ps aux
- Real-time view: top
- Interactive view: htop
- Find process: pgrep
- Kill process: kill <PID>
- Kill forcefully: kill -9 <PID>
- Process priority: nice, renice
- Background process: &
- Jobs: jobs, fg, bg

## 6 SERVICES

- Services are long-running processes
- Examples: sshd, nginx, mysql, docker
- Start service: systemctl start
- Stop service: systemctl stop
- Restart service: systemctl restart
- Check status: systemctl status
- Enable on boot: systemctl enable
- Disable on boot: systemctl disable

## 7 PACKAGE MANAGEMENT

**DEBIAN / UBUNTU**

- Update index: apt update
- Search package: apt search <pkg>
- Install package: apt install <pkg>
- Remove package: apt remove <pkg>
- Upgrade system: apt upgrade -y

**RHEL / CENTOS / ROCKY**

- Update index: yum update (old)
- Install package: yum install <pkg>
- Remove package: yum remove <pkg>
- Search package: yum search <pkg>
- DNF (modern): dnf install <pkg>

## 8 SYSTEMD

- System and service manager for Linux
- Manage services, targets, sockets, timers, etc.
- List all units: systemctl list-units
- List running services: systemctl list-units --type=service --state=running
- Check service: systemctl status <service>
- View logs: journalctl -u <service>
- Service enabled?: systemctl is-enabled <service>
- Reload service: systemctl reload <service>

## 9 LOGS

- Logs are critical for troubleshooting
- View logs: journalctl
- Follow logs: journalctl -f
- Logs for a service: journalctl -u <service> -f
- Boot logs: journalctl -b
- Previous boot logs: journalctl -b -1
- System logs: /var/log/
- Auth logs: /var/log/auth.log
- Error logs: /var/log/syslog

## 10 ESSENTIAL PRODUCTION COMMANDS (QUICK VIEW)

- df -h : Disk usage
- free -h : Memory usage
- uptime : System uptime
- hostnamectl : System info
- ip a or ip addr : IP address
- ss -tulnp : Open ports
- grep -i error /var/log/* : Search errors
- tail -f /var/log/syslog : Live log
- who : Logged in users
- w : User activity

## ESSENTIAL PRODUCTION COMMANDS

### FILE & DIRECTORY

| | |
|---|---|
| ls -lah | # List all files |
| cd /path | # Change directory |
| pwd | # Print working directory |
| mkdir dir | # Create directory |
| rm file | # Remove file |
| rm -rf dir | # Remove directory forcefully |
| cp src dest | # Copy file or directory |
| mv src dest | # Move / rename |
| find /path -name "file" | # Find file |

### SYSTEM INFO

| | |
|---|---|
| uname -a | # Kernel and system info |
| cat /etc/os-release | # OS information |
| hostnamectl | # Hostname and system info |
| uptime | # System uptime |
| lscpu | # CPU information |
| free -h | # Memory usage |
| df -h | # Disk usage |
| du -sh /path | # Directory size |
| top / htop | # Process monitoring |

### NETWORKING

| | |
|---|---|
| ip a | # IP address |
| ip r | # Routing table |
| ss -tulnp | # Listening ports |
| ping host | # Check connectivity |
| traceroute host | # Trace network path |
| dig domain | # DNS lookup |
| curl -I http://host | # HTTP headers |
| ssh user@host | # SSH connection |
| scp file user@host:/path | # Secure copy |

### LOGS & TROUBLESHOOTING

| | |
|---|---|
| journalctl | # All logs |
| journalctl -f | # Follow logs |
| journalctl -u sshd | # Service logs |
| dmesg \| tail | # Kernel messages |
| cat /var/log/syslog | # System logs |
| grep -i error /var/log/* | # Search errors |
| tail -f file | # Follow a file |
| less file | # View file content |
| lsof -i :80 | # Who uses port 80 |

### PERMISSIONS & OWNERSHIP

| | |
|---|---|
| ls -l | # List with permissions |
| chmod 755 file | # Change permissions |
| chown user:group file | # Change owner |
| chgrp group file | # Change group |
| sudo !! | # Repeat last command |
| sudo -i | # Become root |
| passwd user | # Change password |
| id user | # View user and group IDs |

**MASTER LINUX FUNDAMENTALS. IT IS THE FOUNDATION OF EVERY DEVOPS ENGINEER'S DAY-TO-DAY WORK.**

VERIQTA SOCIAL

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

PAGE 3

# VERIQTA
ENGINEERING EXCELLENCE

# 3. LINUX PRODUCTION TROUBLESHOOTING

**DEVOPS BIBLE**
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

Find problems fast. Diagnose accurately. Fix confidently.

## 1 CPU INVESTIGATION

- High CPU usage slows down the system.
- Identify top consumers and root cause.

### KEY COMMANDS

```
top
htop
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu
mpstat -P ALL 1
pidstat -u 1
sar -u 1 5
vmstat 1
```

## 2 MEMORY PRESSURE

- Memory issues cause slowdowns or OOM kills.
- Check usage, cache, swap, and processes.

### KEY COMMANDS

```
free -h
top
htop
vmstat 1
swapon --show
ps -eo pid,ppid,cmd,%mem --sort=-%mem
cat /proc/meminfo
```

## 3 DISK UTILIZATION

- High disk usage impacts performance and stability.
- Find what is using space.

### KEY COMMANDS

```
df -h
du -sh /* 2>/dev/null | sort -h
ncdu /
lsblk
iotop -oPa
iotop -boPa
sar -d 1 5
```

## 4 FILESYSTEM PROBLEMS

- Filesystem errors can cause crashes and data corruption.
- Check and repair safely.

### KEY COMMANDS

```
dmesg | tail
fsck -n /dev/sdX#
mount | column -t
findmnt
umount /mount/point
journalctl -k | tail
```

## 5 PROCESS FAILURES

- Processes may crash, hang, or become zombie.
- Find and fix the root cause.

### KEY COMMANDS

```
ps aux
pstree -p
kill -9 <PID>
killall -9 <process>
pkill -f <pattern>
top
htop
```

## 6 SERVICE FAILURES

- Services may fail to start, stop unexpectedly, or become unhealthy.
- Check status and logs.

### KEY COMMANDS

```
systemctl status <service>
systemctl start <service>
systemctl stop <service>
systemctl restart <service>
systemctl enable <service>
journalctl -u <service> -f
systemctl list-units --failed
```

## 7 LOG INVESTIGATION

- Logs contain the answers.
- Check system, service, and application logs.

### KEY COMMANDS

```
journalctl -xe
journalctl -f
journalctl -u <service>
tail -f /var/log/messages
tail -f /var/log/syslog
grep -i "error" /var/log/*
less /var/log/<file>.log
```

## 8 NETWORK TROUBLESHOOTING

- Network issues cause timeouts, failures, and slow performance.
- Trace and test connectivity.

### KEY COMMANDS

```
ip a
ip r
ss -tulnp
ping <host>
traceroute <host>
dig <domain>
nslookup <domain>
curl -I http://<host>
mtr <host>
```

## 9 PERFORMANCE BOTTLENECKS

- Identify system bottlenecks affecting performance.
- CPU, memory, disk, network, or I/O.

### KEY COMMANDS

```
top
htop
vmstat 1
iostat -xz 1
sar -q 1 5
pidstat 1
iotop -oPa
perf top
```

## 10 PRODUCTION TROUBLESHOOTING WORKFLOW

1. Confirm the problem and scope.
2. Check recent changes (deploys, configs).
3. Collect evidence: metrics, logs, and system info.
4. Form a hypothesis and test it.
5. Identify root cause.
6. Apply safe fix or mitigation.
7. Verify and monitor the resolution.
8. Document the issue and learn.

## QUICK HEALTH CHECK COMMANDS

| SYSTEM OVERVIEW | CPU | MEMORY | DISK | I/O | NETWORK | LOGS |
|---|---|---|---|---|---|---|
| uptime | top | free -h | df -h | iostat -xz 1 | ip a | journalctl -xe |
| w | htop | vmstat 1 | du -sh /* 2>/dev/null \| sort -h | iotop -oPa | ip r | journalctl -u <service> |
| who -a | mpstat -P ALL 1 | cat /proc/meminfo | lsblk | iotop -boPa | ss -tulnp | tail -f /var/log/messages |
| hostnamectl | sar -u 1 5 | | | sar -d 1 5 | ping <host> | |

## TROUBLESHOOTING GOLDEN RULES

- **OBSERVE** — Collect facts, not assumptions.
- **ISOLATE** — Reduce the problem to the smallest possible scope.
- **HYPOTHESIZE** — Form a theory based on evidence.
- **TEST** — Prove or disprove your theory with commands.
- **FIX SAFELY** — Apply the least risky effective change.
- **VERIFY** — Confirm the fix and monitor the results.
- **DOCUMENT** — Record what happened and what you learned.

**A CALM MIND. A CLEAR PROCESS. THE RIGHT COMMANDS. THAT IS HOW PRODUCTION PROBLEMS ARE SOLVED.**

VERIQTA SOCIAL

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

PAGE 4

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 4. NETWORKING
## EVERY DEVOPS ENGINEER MUST KNOW

Understand the network. Troubleshoot with confidence. Build reliable systems.

### 1 TCP/IP

- APPLICATION (Layer 7)
- TRANSPORT (Layer 4)
- INTERNET (Layer 3)
- NETWORK ACCESS (Layer 2)

- TCP/IP is the foundation of all network communication.
- Data travels through layers from application to network access.
- Each layer has a specific role.

### 2 IP ADDRESSES

IPv4 Example: 192.168.1.10

| 192 | 168 | 1 | 10 |
|---|---|---|---|
| Network | Network | Host | Host |

- Unique identifier for devices on a network.
- Two types: IPv4 (32-bit) and IPv6 (128-bit).
- Private IP ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

### 3 CIDR

192.168.1.0/24

IP Address + Prefix Length

24 bits for network | 8 bits for hosts

- CIDR reduces IP waste.
- /24 = 255.255.255.0 (256 addresses)
- More hosts = smaller prefix (e.g. /16).

### 4 SUBNETS

192.168.1.0/24

/24

/26 /26 /26 /26

- Subnetting divides networks into smaller parts.
- Improves performance, security, and organization.
- Helps control broadcast domains.

### 5 ROUTING

10.0.1.0/24 → R → 10.0.2.0/24

ROUTING TABLE

| DESTINATION | NEXT HOP | INTERFACE |
|---|---|---|
| 10.0.1.0/24 | direct | eth0 |
| 10.0.2.0/24 | direct | eth1 |
| 0.0.0.0/0 | 192.168.1.1 | eth0 |

- Routing moves data between networks.
- Routers use routing tables to decide the best path.
- Default route (0.0.0.0/0) is used for unknown destinations.

### 6 DNS

example.com → DNS SERVER → 93.184.216.34

- DNS translates domain names to IP addresses.
- Uses a hierarchical, distributed system.
- Common record types: A, AAAA, CNAME, MX, TXT.
- TTL controls how long records are cached.

### 7 PORTS

0 - 65535

WELL-KNOWN 0 - 1023 | REGISTERED 1024 - 49151

DYNAMIC / PRIVATE 49152 - 65535

- Ports identify applications on a host.
- Examples: 22 (SSH), 80 (HTTP), 443 (HTTPS).
- Use firewall rules to control access.

### 8 TCP vs UDP

| TCP | UDP |
|---|---|
| Connection-Oriented<br>Reliable<br>Ordered<br>Error-Checked | Connectionless<br>Faster<br>No Ordering<br>No Guarantee |
| Use for: Web, Email, File Transfer, SSH | Use for: DNS, VoIP, Streaming, Gaming |

- TCP = Reliability over speed.
- UDP = Speed over reliability.

### 9 NAT

PRIVATE 10.0.0.5 → NAT ROUTER → PUBLIC 203.0.113.10

NAT TABLE (EXAMPLE)

| PRIVATE IP:PORT | PUBLIC IP:PORT |
|---|---|
| 10.0.0.5:52345 | 203.0.113.10:40001 |
| 10.0.0.6:52346 | 203.0.113.10:40002 |

- NAT allows private IPs to access public networks.
- Hides internal networks.
- Types: SNAT, DNAT, PAT (NAT Overload).

### 10 FIREWALLS

- Firewalls control inbound and outbound traffic.
- Stateless: checks packets only.
- Stateful: tracks connection state.
- Rules: Allow / Deny based on IP, Port, Protocol.
- Always follow the principle of least privilege.

### 11 HTTP and HTTPS

HTTP (port 80) → HTTPS (port 443)

- HTTP = Unencrypted communication.
- HTTPS = Encrypted with TLS.
- HTTPS protects data integrity, privacy, and authentication.

### 12 TLS

CLIENT ↔ SERVER

TLS HANDSHAKE

- TLS encrypts data in transit.
- Ensures confidentiality, integrity, and authentication.
- Uses certificates, keys, and secure ciphers.

### 13 LOAD BALANCING

USERS → LOAD BALANCER → BACKEND SERVERS (SERVER 1, SERVER 2, ... SERVER N)

- Distributes traffic across multiple servers.
- Improves availability, performance, and scalability.
- Types: L4 (Transport) and L7 (Application).
- Algorithms: Round Robin, Least Connections, IP Hash.

**STRONG NETWORKING KNOWLEDGE IS THE FOUNDATION OF RELIABLE, SECURE, AND SCALABLE SYSTEMS.**

PAGE 5

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 5. GIT AND PRODUCTION VERSION CONTROL

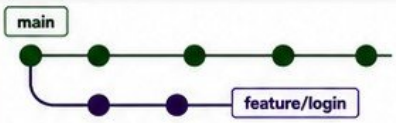
Track changes. Collaborate safely. Ship with confidence.

## 1 REPOSITORY WORKFLOW

WORKING DIRECTORY → STAGING AREA → LOCAL REPOSITORY → REMOTE REPOSITORY

- Make changes in your working directory.
- Stage changes for the next commit.
- Commit changes to your local repository.
- Push commits to the remote repository.

## 2 BRANCHING




- Branches isolate work.
- Create a branch for new features or fixes.
- Merge changes back to main when ready.

**KEY COMMANDS**

```
git branch
git branch <name>
git switch <name>
git switch -c <name>
```

## 3 COMMITS

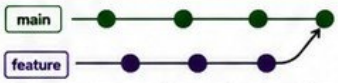



- Commits capture snapshots of changes.
- Write clear, meaningful commit messages.
- Each commit has a unique ID (SHA).

**KEY COMMANDS**

```
git status
git add <file>
git commit -m "message"
git log --oneline --graph
```

## 4 MERGE

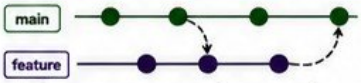



- Merge integrates changes from one branch into another.
- Creates a merge commit by default.

**KEY COMMANDS**

```
git merge <branch>
git merge --no-ff <branch>
git log --graph --oneline --all
```

## 5 REBASE




- Rebase moves your commits on top of another branch.
- Keeps history clean and linear.

**KEY COMMANDS**

```
git rebase <branch>
git rebase -i <branch>
git rebase --abort
git rebase --continue
```

## 6 PULL REQUESTS

- Propose changes from a branch.
- Discuss, review, and iterate.
- Merge after approval and CI checks.

**KEY COMMANDS**

```
git push origin <branch>
(Create PR on GitHub/GitLab)
git fetch origin
git pull origin <branch>
```

## 7 CODE REVIEWS

- Ensure code quality and consistency.
- Share knowledge and reduce bugs.
- Reviews before merge, not after.

**BEST PRACTICES**

- ✅ Focus on code, not the person.
- ✅ Keep comments clear and constructive.
- ✅ Approve only when requirements are met.

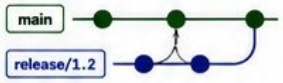
## 8 TAGS

- Tags mark important points in history (releases, versions).
- Immutable references.

**KEY COMMANDS**

```
git tag
git tag -a v1.0.0 -m "v1.0.0 release"
git push origin v1.0.0
```

## 9 RELEASE BRANCHES




- Stabilize features before production.
- Allow bug fixes without new features.
- Merge back to main after release.

**KEY COMMANDS**

```
git switch -c release/1.2
git merge release/1.2
git branch -d release/1.2
```

## 10 BRANCH PROTECTION

- Protect important branches (main).
- Require reviews and status checks.
- Prevent force push and deletions.

**COMMON RULES**

- ✅ Require pull request reviews
- ✅ Require status checks to pass
- ✅ Require branches up to date
- ✅ Restrict force pushes
- ✅ Restrict branch deletions

## 11 GIT TROUBLESHOOTING

| PROBLEM | CAUSE | SOLUTION | COMMANDS |
|---|---|---|---|
| Accidental changes not committed | Changes still in working directory | Discard or stash changes | `git restore <file>`<br>`git stash` |
| Committed to wrong branch | Commit made on wrong branch | Move commit to correct branch | `git switch <branch>`<br>`git cherry-pick <commit>` |
| Need to undo last commit | Wrong commit or message | Undo commit, keep changes | `git reset --soft HEAD~1` |
| Pushed wrong commit | Bad commit already pushed | Revert commit safely | `git revert <commit>` |
| Merge conflicts | Changes conflict between branches | Resolve conflicts manually | `git status`<br>`git add <resolved files>`<br>`git commit` |
| Detached HEAD | Checked out a commit instead of branch | Switch back to branch | `git switch <branch>` |
| Lost commits | Commits not reachable | Use reflog to find them | `git reflog` |

## DAILY GIT COMMAND CHEAT SHEET

| | |
|---|---|
| `git status` | Show change status |
| `git add .` | Stage all changes |
| `git commit -m "msg"` | Commit changes |
| `git log --oneline --graph` | View commit history |
| `git branch` | List branches |
| `git switch <branch>` | Switch branch |
| `git switch -c <branch>` | Create & switch branch |
| `git merge <branch>` | Merge branch |
| `git pull` | Fetch and merge |
| `git push` | Push changes |
| `git fetch --all` | Fetch all remotes |
| `git tag` | List tags |
| `git stash` | Stash changes |

GOOD VERSION CONTROL IS THE FOUNDATION OF COLLABORATION, QUALITY, AND RELIABLE DELIVERIES.

VERIQTA SOCIAL
YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

DEVOPS BIBLE

# VERIQTA
ENGINEERING EXCELLENCE

FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

PAGE 6

# CLOUD INFRASTRUCTURE FUNDAMENTALS

THE BUILDING BLOCKS OF MODERN CLOUD PLATFORMS

## 1 COMPUTE

- Virtual machines and containers run your applications.
- Scale up or down based on demand.
- Pay for what you use.
- Examples: VM, Containers, Serverless Functions.

## 2 STORAGE

- Store and retrieve data durably and securely.
- Different types for different needs.
- Scalable, reliable, and highly available.
- Examples: Object Storage, Block Storage, File Storage.

## 3 NETWORKING

- Connect resources securely and efficiently.
- Includes VPC, subnets, IP addressing, routing.
- Enable communication within and outside the cloud.
- Examples: VPC, DNS, VPN, Gateways.

## 4 DATABASES

- Managed database services for reliability and scale.
- Relational and NoSQL options available.
- Automated backups and replication.
- Examples: RDS, Aurora, DynamoDB, Cloud SQL.

## 5 IDENTITY

- Manage users, roles, and permissions securely.
- Control access to cloud resources.
- Use least privilege principle.
- Examples: IAM, Roles, Policies, MFA.

## 6 LOAD BALANCING

- Distribute traffic across multiple resources.
- Improve availability and application performance.
- Health checks ensure only healthy targets receive traffic.
- Examples: ALB, NLB, Gateway LB.

## 7 AUTO SCALING

- Automatically adjust capacity based on demand.
- Ensure performance and optimize cost.
- Scale out or in based on metrics or schedules.
- Examples: Auto Scaling Groups, Scale Sets.

## 8 AVAILABILITY ZONES

- Physically separate data centers within a region.
- Isolate failures and improve uptime.
- Deploy resources across multiple AZs.
- Examples: AZ1, AZ2, AZ3.

## 9 REGIONS

- Geographically separate areas of the world.
- Choose region close to your users.
- Used for compliance, latency, and DR.
- Examples: us-east-1, eu-west-1, ap-southeast-1.

## 10 SHARED RESPONSIBILITY

- Cloud provider secures the cloud.
- You secure what you build in the cloud.
- Understand your responsibilities.
- Key to a secure cloud environment.

## 11 HIGH AVAILABILITY

- Design systems to remain available during failures.
- Use multi-AZ, load balancing, and auto scaling.
- Eliminate single points of failure.
- Regularly test and validate your architecture.
- Goal: Maximize uptime and reliability.

## KEY TAKEAWAYS

- ✓ Cloud provides scalable, reliable, and cost-effective services.
- ✓ Understand each building block and how they work together.
- ✓ Design for security, reliability, and performance from day one.
- ✓ Automate, monitor, and continuously optimize.
- ✓ Choose the right service for the right problem.
- ✓ Always plan for failure and recovery.

## HOW THESE SERVICES WORK TOGETHER

COMPUTE → STORAGE → NETWORKING → DATABASES → IDENTITY → LOAD BALANCING → AUTO SCALING → AZs → HIGH AVAILABILITY

BUILD ► CONNECT ► SECURE ► SCALE ► DELIVER

## REMEMBER

Great cloud foundations lead to great applications.

Design smart.
Build secure.
Scale with confidence.

## VERIQTA SOCIAL

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

DEVOPS BIBLE

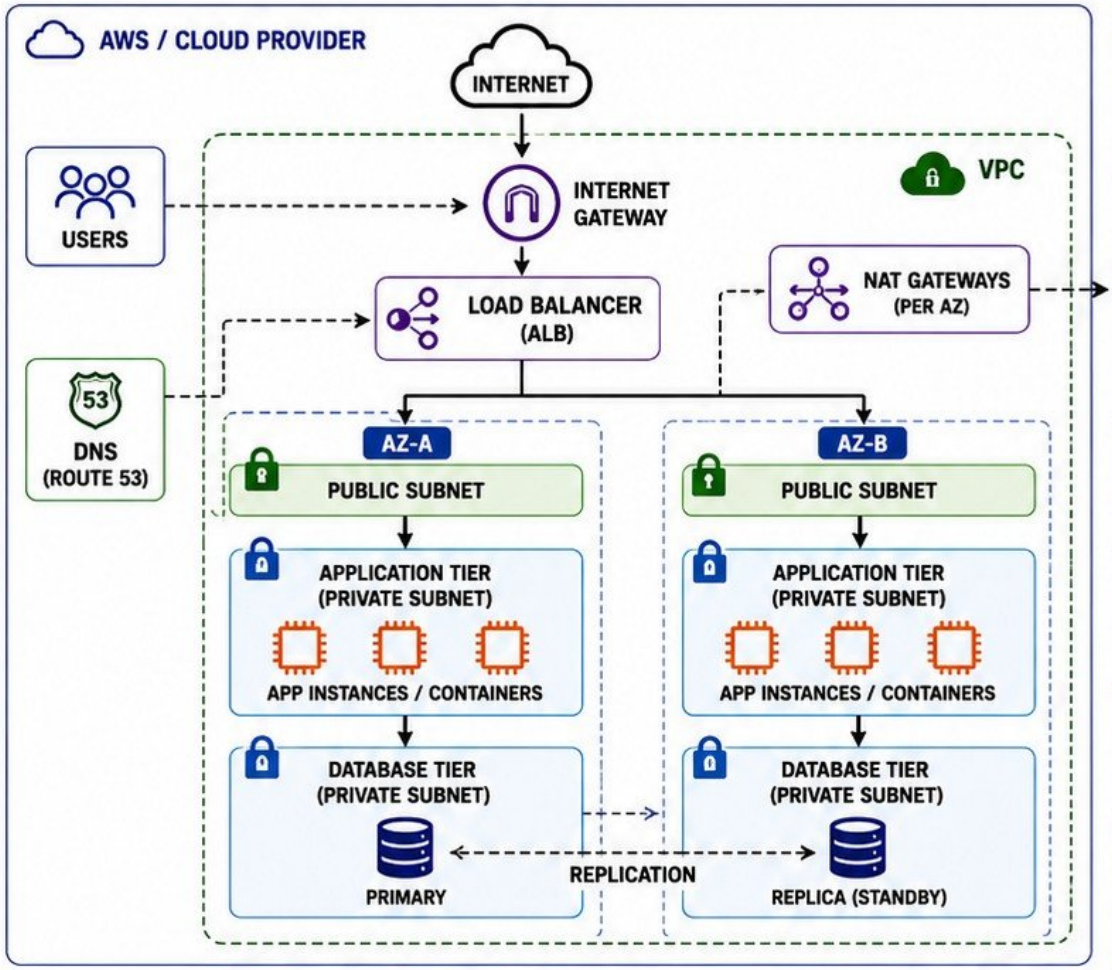
# VERIQTA
ENGINEERING EXCELLENCE

FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

PAGE 7

# PRODUCTION CLOUD ARCHITECTURE

DESIGN RESILIENT, SCALABLE, AND HIGHLY AVAILABLE CLOUD SYSTEMS




## CORE COMPONENTS

**PUBLIC & PRIVATE SUBNETS**
Segment and isolate resources securely.

**INTERNET GATEWAY**
Connects the VPC to the internet.

**NAT GATEWAY**
Allows private subnet resources to access the internet securely.

**LOAD BALANCER**
Distributes traffic across healthy application instances.

**APPLICATION TIERS**
Run stateless compute workloads that scale.

**DATABASE TIERS**
Store critical data with replication across AZs.

**MULTI-AZ ARCHITECTURE**
Ensures high availability and fault tolerance.

## KEY ARCHITECTURE PRINCIPLES

**FAULT ISOLATION**
Isolate failures so issues in one part do not affect others.

**SCALING**
Scale application tiers horizontally based on demand.

**RESILIENCE**
Design for failure. Systems recover automatically and quickly.

**HIGH AVAILABILITY**
Use multiple AZs and remove single points of failure.

**SECURITY**
Follow the principle of least privilege and defense in depth.

## PRODUCTION ARCHITECTURE PATTERNS

- 3-TIER ARCHITECTURE
- WEB APP ARCHITECTURE
- MICROSERVICES ARCHITECTURE
- MULTI-AZ HIGH AVAILABILITY
- ACTIVE-PASSIVE (DR) ARCHITECTURE
- ACTIVE-ACTIVE (MULTI-REGION)
- SERVERLESS ARCHITECTURE
- HYBRID CLOUD ARCHITECTURE

## WHAT THIS ARCHITECTURE DELIVERS

HIGH PERFORMANCE | MAXIMUM AVAILABILITY | ELASTIC SCALING | SECURE BY DESIGN | COST OPTIMIZED | DISASTER RESILIENT

## REMEMBER

A strong architecture today prevents incidents tomorrow. Design for scale, security, and reliability from day one.

## VERIQTA SOCIAL

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

DEVOPS BIBLE

# VERIQTA
ENGINEERING EXCELLENCE

FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

PAGE 8

# INFRASTRUCTURE AS CODE

BUILD, MANAGE, AND SCALE INFRASTRUCTURE WITH CODE

## 1 WHY INFRASTRUCTURE SHOULD BE CODE

- Version controlled and auditable
- Consistent and repeatable deployments
- Automated and faster provisioning
- Easy to scale and modify
- Real-time collaboration
- Reduced manual errors
- Disaster recovery and restore
- Enables CI/CD for infrastructure
- Cost-effective and efficient
- Infrastructure becomes a product

## 2 DECLARATIVE INFRASTRUCTURE

DESIRED STATE → TERRAFORM RECONCILES THE STATE → REAL INFRASTRUCTURE

**YOU DEFINE THE DESIRED STATE, TERRAFORM ACHIEVES IT.**

## 3 TERRAFORM FUNDAMENTALS

### 3.1 PROVIDERS

Providers are plugins that interact with APIs of cloud or services.
Example: aws, azurerm, google, kubernetes.

### 3.2 RESOURCES

Resources are the infrastructure objects you create and manage.
Example: aws_instance, aws_s3_bucket.

### 3.3 VARIABLES

Variables make your code dynamic and reusable.
Example: region, instance_type.

### 3.4 OUTPUTS

Outputs return important values after apply.
Example: instance IP, endpoint, DNS name.

### 3.5 MODULES

Modules organize and reuse code.
Build once, use everywhere.

### 3.6 STATE

State is Terraform's memory of your infrastructure.
It maps config to real resources.

### 3.7 REMOTE STATE

Store state remotely for collaboration, locking, and safety.
Example: S3 + DynamoDB, Azure Storage.

### 3.8 INFRASTRUCTURE LIFECYCLE

Plan → Apply → Change → Destroy
Manage infrastructure through its full lifecycle.

### 3.9 DEPENDENCIES

Terraform automatically understands dependencies between resources.
Creates in correct order.

### 3.10 IDEMPOTENCY

Apply multiple times, same result.
No changes if infrastructure matches desired state.

## 4 INFRASTRUCTURE LIFECYCLE WITH TERRAFORM

| WRITE CODE | PLAN | APPLY | MANAGE | DESTROY |
|---|---|---|---|---|
| Define desired infrastructure in .tf files | Review the execution plan (terraform plan) | Apply the changes to provision or update resources | Terraform tracks and manages the state | Remove resources when no longer needed |

## 5 BENEFITS OF INFRASTRUCTURE AS CODE

- Consistency across environments
- Safe collaboration for teams
- Automated infrastructure provisioning
- Faster recovery and disaster readiness
- Lower operational cost
- Infrastructure is self-documenting
- Enables DevOps and GitOps practices
- Better security and compliance

**INFRASTRUCTURE AS CODE IS THE FOUNDATION OF MODERN DEVOPS.**
**CODE IT. VERSION IT. AUTOMATE IT. SCALE IT.**

VERIQTA SOCIAL
YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

DEVOPS BIBLE

# VERIQTA
ENGINEERING EXCELLENCE

FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

PAGE 9

# PRODUCTION TERRAFORM

BUILD, MANAGE, AND OPERATE INFRASTRUCTURE AT SCALE

## 1 MODULE ARCHITECTURE

- Organize code into reusable modules.
- Follow DRY (Don't Repeat Yourself).
- Keep modules small and focused.
- Use consistent inputs and outputs.
- Publish and version modules.
- Example structure:

```
modules/
├── vpc/
├── subnet/
├── security-group/
├── eks/
└── rds/
```

## 2 ENVIRONMENT SEPARATION

DEV — STAGE — PROD

- Separate environments with folders or workspaces.
- Use separate state for each environment.
- Different variables per environment.
- Example folder structure:

```
environments/
├── dev/
│   └── main.tf
├── stage/
│   └── main.tf
└── prod/
    └── main.tf
```

## 3 STATE LOCKING

- Prevent concurrent changes to state.
- Use state locking to avoid corruption.
- Common backends provide built-in locking.
- Always enable locking.

## 4 REMOTE BACKENDS

- Store state remotely for safety.
- Enable locking and versioning.
- Common backends:

aws Amazon S3 | Azure Storage | Google Cloud Storage

## 5 DEPENDENCY MANAGEMENT

- Use terraform dependency blocks when needed.
- Use data sources to reference existing resources.
- Explicit dependencies improve reliability.
- Run `terraform init` to install providers and modules.

## 6 TERRAFORM PLAN

- Preview changes before apply.
- Detects what will be created, updated, or destroyed.
- Use in CI/CD for validation.
- Command:

```
terraform plan
```

## 7 TERRAFORM APPLY

- Apply the approved changes.
- Updates the real infrastructure to match the desired state.
- Use `-auto-approve` in automation (with caution).
- Command:

```
terraform apply
```

## 8 DRIFT DETECTION

- Drift happens when real infrastructure differs from state.
- Use terraform `plan` to detect drift.
- Integrate drift detection in pipelines and monitoring.
- Regularly audit critical resources.

## 9 IMPORTING RESOURCES

- Bring existing resources under Terraform management.
- Use terraform import.
- Example:

```
terraform import
aws_s3_bucket.bucket my-bucket
```

## 10 SECURITY

- Follow least privilege IAM.
- Store state securely.
- Encrypt state at rest.
- Do not commit secrets.
- Use variable validation.
- Scan code for security issues.
- Use secrets managers.

## 11 CI/CD INTEGRATION

CODE COMMIT → INIT terraform init → VALIDATE terraform validate → PLAN terraform plan → APPLY terraform apply → MONITOR monitor & alert

- Automate terraform `init`, plan, and apply.
- Use plan in pull requests for review.
- Require approvals for apply to protected environments.
- Store state securely in remote backend.
- Add notifications and failure alerts.
- Ensure audit and logging.

## BEST PRACTICES

- Keep modules reusable and versioned.
- Separate environments and states.
- Always review plans before apply.
- Protect and backup state files.
- Automate with CI/CD pipelines.
- Monitor, audit, and continuously improve.

**INFRASTRUCTURE AS CODE IS NOT JUST AUTOMATION.**
**IT IS RELIABILITY, REPEATABILITY, AND CONTROL AT SCALE.**


VERIQTA SOCIAL

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

DEVOPS BIBLE

PAGE 10

VERIQTA
ENGINEERING EXCELLENCE

FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

ANSIBLE

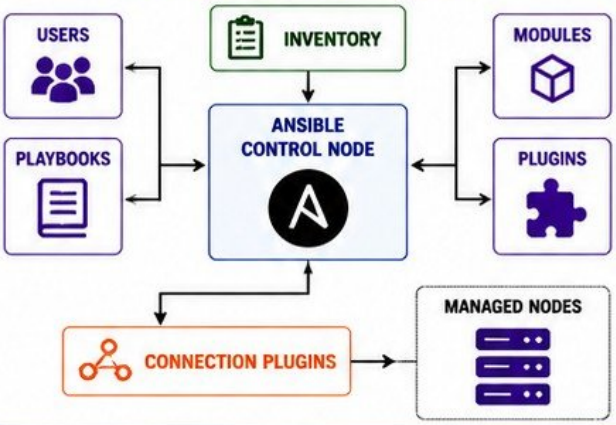
# CONFIGURATION MANAGEMENT

AUTOMATE, STANDARDIZE, AND MAINTAIN INFRASTRUCTURE AT SCALE

## 1 CONFIGURATION MANAGEMENT CONCEPTS

- Ensure consistency across systems.
- Automate system configuration and management.
- Reduce manual work and errors.
- Enforce desired state.
- Support scalability and compliance.
- Track and audit changes.
- Improve reliability and security.

## 2 ANSIBLE ARCHITECTURE




## 3 INVENTORY

- Define the hosts Ansible manages.
- Organize hosts into groups.
- Static inventory or dynamic inventory.
- Use INI, YAML, or scripts.
- Example:

```
[web]
 web1.example.com
 web2.example.com

[db]
 db1.example.com
```

## 4 PLAYBOOKS

- Define automation tasks in YAML.
- Sequence of tasks.
- Easy to read and version.
- Reusable and shareable.
- Example:

```
- hosts: web
  become: yes
  tasks:
  - name: Install Nginx
    apt:
      name: nginx
      state: present
```

## 5 ROLES

- Organize playbooks into reusable roles.
- Standard structure: tasks, handlers, templates, files, vars, defaults, meta.
- Promote reusability and consistency.

Example:

```
roles/
└── webserver/
    ├── tasks/
    ├── templates/
    ├── files/
    ├── vars/
    └── defaults/
```

## 6 VARIABLES

- Store values for reuse.
- Use in playbooks, roles, and templates.
- Variable precedence matters.
- Sources: vars, defaults, extra-vars, inventory.
- Example:

```
app_port: 8080
app_user: deploy
app_env: production
```

## 7 TEMPLATES

- Generate dynamic config files.
- Use Jinja2 templating engine.
- Insert variables into templates.
- Ensure consistency across systems.

Example:

```
server {
 listen {{ app_port }};
 server_name {{ domain }};
}
```

## 8 IDEMPOTENCY

- Ansible ensures tasks are idempotent.
- Running multiple times has the same result.
- Prevents drift and unnecessary changes.
- Use idempotent modules.
- Example: apt, copy, user.

## 9 SECRETS

- Never store plain secrets in playbooks.
- Use Ansible Vault to encrypt secrets.
- Use external secret stores when possible.
- Keep secrets out of version control.
- Example:

```
ansible-vault encrypt secrets.yml
```

## 10 SERVER CONFIGURATION

- Install and configure packages.
- Manage users and permissions.
- Configure services.
- Deploy and update applications.
- Manage files and directories.
- Enforce security baselines.
- Ensure compliance.

## 11 CONFIGURATION DRIFT

- Drift happens when systems change outside automation.
- Detect drift using Ansible runs.
- Use Ansible to revert to desired state.
- Schedule regular audits.
- Maintain system consistency.

## KEY BENEFITS

- Consistency across environments.
- Faster provisioning and updates.
- Reduced manual errors.
- Improved security and compliance.
- Scalable and repeatable operations.

## CONFIGURATION MANAGEMENT WORKFLOW

DEFINE DESIRED STATE → WRITE PLAYBOOKS → RUN ANSIBLE → APPLY CHANGES TO NODES → VERIFY & MAINTAIN COMPLIANCE

## REMEMBER

Automate the boring.
Standardize everything.
Enforce the desired state.
Repeatable systems build reliable platforms.

**CONFIGURATION MANAGEMENT IS THE FOUNDATION OF RELIABLE INFRASTRUCTURE.**
**AUTOMATE TODAY. OPERATE BETTER TOMORROW.**

VERIQTA SOCIAL

YouTube veriqta | GitHub veriqta | LinkedIn veriqta | X veriqta | Instagram veriqta | Telegram veriqta

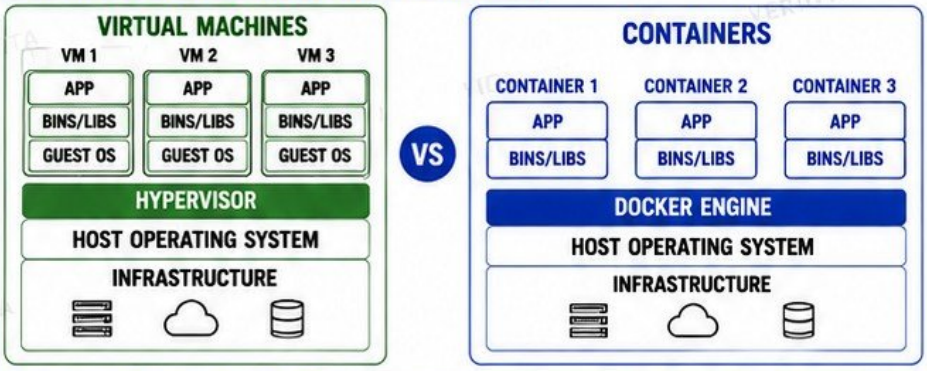
VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
40-PAGE HANDBOOK

# 11. DOCKER FUNDAMENTALS

Build. Ship. Run. Anywhere.

## 1. CONTAINERS vs VIRTUAL MACHINES




Containers are lightweight, start fast, and share the host OS kernel.

## 2. IMAGES

- A read-only template.
- Contains everything needed to run an application.
- Built from a Dockerfile.
- Versioned and immutable.
- Stored in registries.

**EXAMPLE**

```
nginx:latest
redis:7
myapp:v1.0.0
```

## 3. CONTAINERS

- A running instance of an image.
- Isolated process environment.
- Can start, stop, move, or remove.
- Has its own writable layer.
- Can be created from images.

## 4. DOCKERFILES

- A text file with instructions.
- Defines how to build an image.
- Uses commands like FROM, COPY, RUN, CMD, EXPOSE.
- Enables automation and consistency.

**EXAMPLE DOCKERFILE**

```
FROM nginx:alpine
COPY ./html /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

## 5. LAYERS

- Images are built in layers.
- Each instruction adds a layer.
- Layers are cached and reused.
- Only changed layers are rebuilt.
- Makes builds fast and efficient.

## 6. REGISTRIES

- Store and distribute images.
- Public: Docker Hub
- Private: ECR, GCR, ACR, Harbor, etc.
- Pull images to run containers.
- Push images to share.

## 7. VOLUMES

- Persist data outside containers.
- Survive container restarts and deletions.
- Types: Named Volumes, Bind Mounts, tmpfs.
- Essential for databases, uploads, logs, and more.

## 8. NETWORKS

- Enable communication between containers and outside world.
- Types: bridge (default), host, overlay, none, macvlan.
- Each network provides isolation and connectivity.

## 9. ENVIRONMENT VARIABLES

- Key-value pairs passed to containers.
- Configure application behavior.
- Set at build time (ENV) or run time (-e).
- Keep images reusable and config flexible.

## 10. CONTAINER LIFECYCLE

- Create → Start → Run
- Stop → Pause → Unpause
- Restart → Kill
- Remove
- Inspect → Logs → Exec
- Commit (rarely used)

### CONTAINER LIFECYCLE FLOW




### KEY TAKEAWAY

Docker gives you a consistent, portable, and efficient way to package and run applications.
Master these fundamentals to build reliable and scalable DevOps solutions.

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
40-PAGE HANDBOOK

# 12. PRODUCTION DOCKER

Build secure. Run efficient. Operate at scale.

## 1. MULTI-STAGE BUILDS

```
# Build-stage
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o app

# Runtime stage
FROM alpine:3.19
WORKDIR /app
COPY --from=builder /app/app .
CMD ["./app"]
```

- Use multi-stage builds.
- Keep final image small.
- Exclude build tools and cache.
- Improves security and performance.

## 2. SMALL IMAGES

- Use minimal base images (Alpine, Distroless, Debian-slim).
- Remove unnecessary packages.
- Reduce layers.
- Clean package manager cache.
- Smaller images = faster, safer, and cheaper.

### GOOD BASE IMAGES

```
alpine:latest
debian:bookworm-slim
ubuntu:22.04
gcr.io/distroless/base
scratch (for static binaries)
```

## 3. IMAGE TAGGING

- Use consistent tagging strategy.
- Use semantic versioning.
- Use immutable tags.
- Avoid "latest" in production.
- Include build metadata if needed.

### EXAMPLES

```
myapp:1.0.0
myapp:1.0.0-rc1
myapp:1.0.0-build.15
myapp:2024-05-10
myapp:sha-4f3a1b2
```

## 4. IMAGE SCANNING

- Scan images for vulnerabilities.
- Use tools like Trivy, Grype, or Docker Scout.
- Fail builds on high/critical vulnerabilities.
- Scan base images regularly.

### EXAMPLE (TRIVY)

```
trivy image myapp:1.0.0
trivy image --severity HIGH,CRITICAL
  --exit-code 1 myapp:1.0.0
```

## 5. NON-ROOT CONTAINERS

- Do not run containers as root.
- Create a dedicated user.
- Drop unnecessary capabilities.
- Use USER in Dockerfile.
- Improves security.

### EXAMPLE

```
RUN adduser -D -g '' appuser
USER appuser
WORKDIR /app
```

## 6. RESOURCE LIMITS

- Limit CPU and memory usage.
- Prevent noisy neighbor issues.
- Protect the host from overload.
- Set limits and requests.

### EXAMPLE (RUN)

```
docker run -d \
  --name myapp \
  --cpus="1.0" \
  --memory="512m" \
  --pids-limit=100 \
  myapp:1.0.0
```

## 7. HEALTH CHECKS

- Detect application failures early.
- Docker can restart unhealthy containers.
- Use HEALTHCHECK in Dockerfile or in run command.

### EXAMPLE (DOCKERFILE)

```
HEALTHCHECK --interval=30s \
--timeout=5s --retries=3 \
CMD curl -f http://localhost:8080/
health || exit 1
```

## 8. PERSISTENT STORAGE

- Containers are ephemeral.
- Use volumes for persistent data.
- Use bind mounts or named volumes.
- Backup important data.

### EXAMPLES

```
docker run -d \
  -v mydata:/data \
  myapp:1.0.0

# Bind mount
docker run -d \
  -v /opt/data:/data \
  myapp:1.0.0
```

## 9. CONTAINER NETWORKING

- Use user-defined bridge networks.
- Enable service discovery.
- Isolate applications.
- Use overlay networks for multi-host setups.
- Avoid exposing unnecessary ports.

### EXAMPLE (NETWORK)

```
docker network create \
  --driver bridge mynet

docker run -d --name web \
  --network mynet myapp:1.0.0
```

## 10. DOCKER TROUBLESHOOTING

- Check container status.
- Inspect logs.
- Inspect container details.
- Check resource usage.
- Verify networks and volumes.
- Common issues: ports, permissions, disk, memory, logs.

### USEFUL COMMANDS

```
docker ps -a
docker logs <container>
docker inspect <container>
docker stats
docker system df
docker system prune -a
```

## COMMON PRODUCTION DOCKER COMMANDS

| | | | | | | |
|---|---|---|---|---|---|---|
| `docker build -t myapp:1.0.0 .` Build image | `docker run -d myapp:1.0.0` Run container | `docker ps` List running containers | `docker logs <container>` View logs | `docker inspect <container>` Inspect details | `docker stats` Resource usage | `docker rm -f <container>` Remove container |

## KEY TAKEAWAY

Production Docker is about security, efficiency, reliability, and operational excellence.
Follow these best practices to build and run containers that are fast, safe, and production-ready.

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
40-PAGE HANDBOOK

# 13. KUBERNETES ARCHITECTURE

Understand the building blocks that run your containers at scale.

## KUBERNETES ARCHITECTURE OVERVIEW

### CONTROL PLANE

**API SERVER**
Front-end for the Kubernetes control plane.

**ETCD**
Consistent and highly-available key-value store for all cluster data.

**SCHEDULER**
Watches for new Pods and selects the best Node for them.

**CONTROLLER MANAGER**
Runs controllers that regulate the state of the cluster.

### NODES (WORKER MACHINES)

**NODE 1**

**KUBELET**
Agent that runs on each node. Ensures containers are running in a Pod.

**KUBE-PROXY**
Network proxy that maintains network rules on the node.

NODS

**NODE 2**

**KUBELET**
Agent that runs on each node. Ensures containers are running in a Pod.

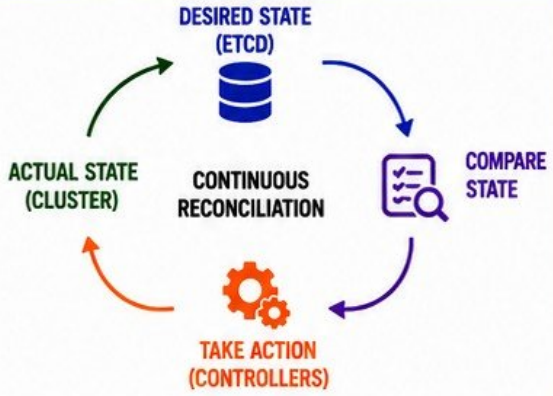
**KUBE-PROXY**
Network proxy that maintains network rules on the node.

PODS

## HOW IT WORKS

1. Users or tools interact with the API Server.
2. API Server validates the request and stores data in etcd.
3. Scheduler watches for new Pods and selects a Node.
4. Kubelet on the Node pulls the Pod spec and runs containers.
5. Kube-proxy configures networking for the Pod and Services.
6. Controllers continuously watch and ensure the desired state.

## KUBERNETES RECONCILIATION LOOP




## KEY BENEFITS

- Highly available and scalable
- Self-healing
- Automatic scheduling
- Desired state management
- Fault tolerant
- Declarative configuration

## COMMUNICATION FLOW




### 1. CONTROL PLANE
The brain of the cluster. Manages the cluster, makes global decisions, and detects cluster events.

### 2. API SERVER
The single entry point for all administrative tasks. Exposes the Kubernetes API.

### 3. ETCD
Stores all cluster data: nodes, pods, configs, secrets, state, and more. It is the source of truth.

### 4. SCHEDULER
Chooses the best node for newly created pods based on resources, policies, and constraints.

### 5. CONTROLLER MANAGER
Runs controllers like Deployment, ReplicaSet, Node, Endpoints, and more to maintain the desired state.

### 6. KUBELET
Ensures containers defined in Pods are running and healthy on the node.

### 7. KUBE-PROXY
Handles networking and load balancing for Services on the node.

### 8. NODES
Worker machines that run your applications (containerized workloads).

### 9. PODS
The smallest deployable units in Kubernetes. One or more containers run together in a Pod.

### 10. KUBERNETES RECONCILIATION
Kubernetes continuously compares the desired state (in etcd) with the actual state (in the cluster) and takes action to make them match.

## KEY TAKEAWAY

Kubernetes is built on a powerful control plane that makes smart decisions and a group of worker nodes that run your containers. The reconciliation loop ensures your applications always run as expected.

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
40-PAGE HANDBOOK

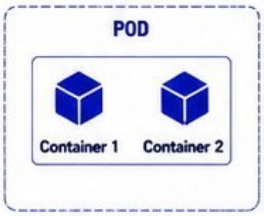
# 14. KUBERNETES WORKLOADS

Run. Scale. Heal. Repeat.

**KUBERNETES WORKLOADS ARE THE BUILDING BLOCKS THAT DEPLOY AND MANAGE YOUR APPLICATIONS.**

## 1. PODS

- Smallest deployable unit in Kubernetes.
- One or more containers that share the same network and storage.
- Ephemeral by nature.

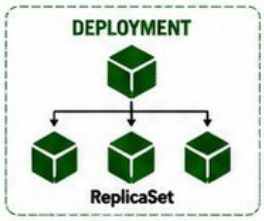



**USE CASES**

- Single container apps
- Sidecar containers
- Tightly coupled containers

## 2. DEPLOYMENTS

- Manages ReplicaSets.
- Provides declarative updates for Pods.
- Supports rolling updates and rollbacks.
- Ensures desired state.

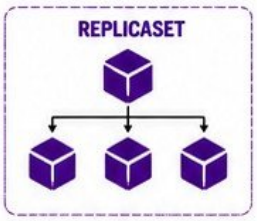



**USE CASES**

- Stateless applications
- Web apps
- APIs and microservices

## 3. REPLICASETS

- Ensures a specified number of Pod replicas are running.
- Replaces failed Pods.
- Usually managed by Deployments.

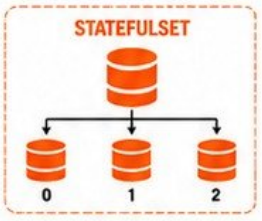



**USE CASES**

- Maintain replica count
- High availability
- Self-healing applications

## 4. STATEFULSETS

- Manages stateful applications.
- Provides stable network identities and storage.
- Ordered deployment and scaling.

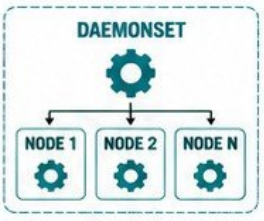



**USE CASES**

- Databases
- Message queues
- Stateful applications

## 5. DAEMONSETS

- Runs a Pod on all (or some) nodes.
- Keeps the Pod running on node additions.
- Useful for node-level tasks.




**USE CASES**

- Logging agents
- Monitoring agents
- Network plugins

## 6. JOBS

- Runs a Pod to completion.
- Ensures the task succeeds.
- Does not restart the Pod on success.

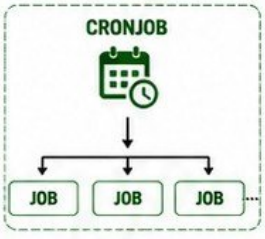



**USE CASES**

- Batch processing
- One-time tasks
- Data migration

## 7. CRONJOBS

- Runs Jobs on a schedule.
- Uses cron syntax for timing.
- Automates recurring tasks.




**USE CASES**

- Backups
- Data cleanup
- Scheduled reports

## 8. ROLLING UPDATES

- Update Pods gradually without downtime.
- Old Pods are replaced with new ones.
- Ensures availability during updates.

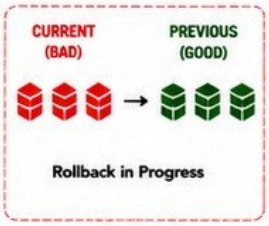



**USE CASES**

- Zero-downtime deployments
- Safe version upgrades
- Continuous delivery

## 9. ROLLBACKS

- Revert to a previous ReplicaSet or version.
- Quick recovery from bad deployments.
- Ensures minimal downtime.

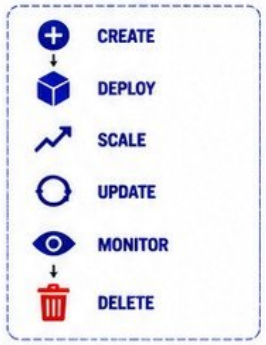



**USE CASES**

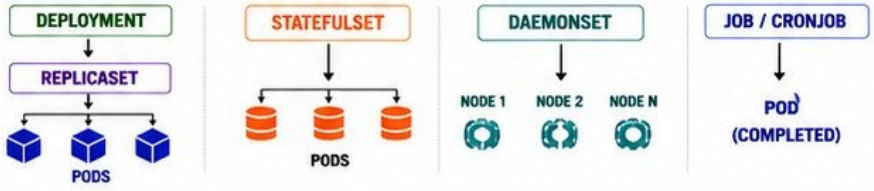
- Failed deployments
- Bug fixes
- Quick recovery

## 10. APPLICATION LIFECYCLE

- Defines the full journey of an application.
- From creation to updates and retirement.
- Ensures reliability and consistency.




## WORKLOAD RELATIONSHIP




## KEY BENEFITS OF KUBERNETES WORKLOADS

- Auto healing and self recovery
- High availability and scalability
- Declarative and consistent deployments
- Efficient resource utilization
- Supports a wide range of application types
- Reliable updates and rollbacks
- Automated and scheduled operations

## KEY TAKEAWAY

Kubernetes workloads help you run, scale, update, and manage applications reliably.
Choose the right workload for the right job.

VERIQTA
ENGINEERING EXCELLENCE

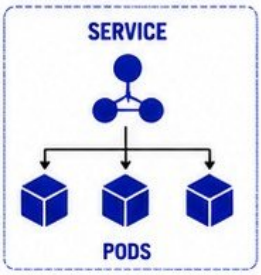
DEVOPS BIBLE
40-PAGE HANDBOOK

# 15. KUBERNETES NETWORKING AND TRAFFIC

Connect. Secure. Discover. Deliver.

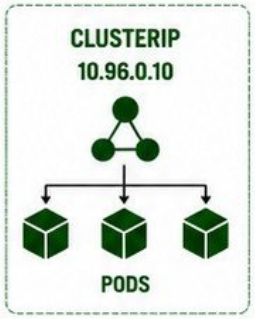
## 1. SERVICES (OVERVIEW)

- Stable virtual IP and DNS for a set of Pods.
- Decouples clients from Pods.
- Load balances traffic across healthy Pods.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.



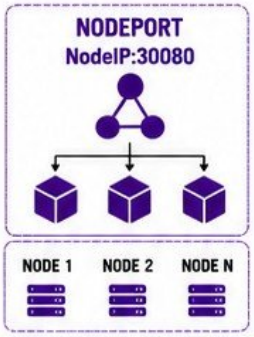

## 2. CLUSTERIP

- Default service type.
- Accessible only within the cluster.
- Stable virtual IP.
- Ideal for internal communication.



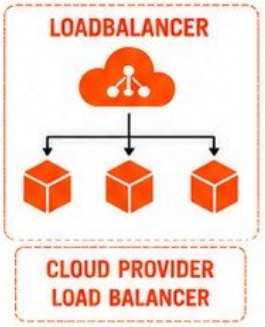

## 3. NODEPORT

- Exposes the service on a static port on each node's IP.
- Accessible from outside the cluster.
- Port range: 30000–32767.



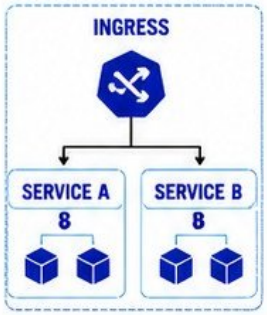

## 4. LOADBALANCER

- Provisions an external load balancer.
- Internet-facing access to the service.
- Cloud provider integration.




## 5. INGRESS

- HTTP/HTTPS traffic routing to services.
- Host and path based routing.
- TLS termination.
- Single entry point.




## 6. DNS

- Kubernetes assigns a DNS name to Services.
- Format:
  `<service>.<namespace>.svc.cluster.local`
- Enables easy service communication.

**EXAMPLE**

frontend.default.svc.cluster.local

↓

POD

Resolves to Service IP

## 7. NETWORK POLICIES

- Control traffic between Pods.
- Allow or deny traffic based on rules.
- Applied at the namespace or Pod level.
- Enforced by CNI plugin.

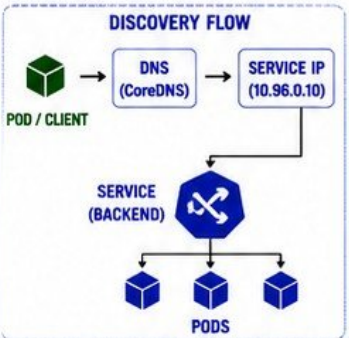



**EXAMPLE RULE**

Allow Pod A (app=frontend) to access Pod B (app=backend) on port 80 only.

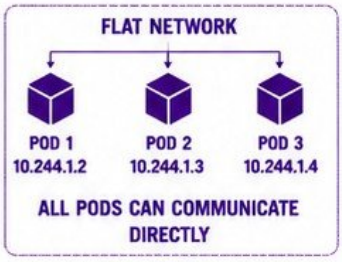
## 8. SERVICE DISCOVERY

- Services get a stable DNS name.
- Clients discover services automatically.
- No hardcoded IPs.
- Works across namespaces.




## 9. CONTAINER NETWORKING

- Pods get IPs from the cluster network.
- Flat network model.
- All Pods can communicate without NAT.
- Implemented by CNI plugins (Calico, Cilium, Flannel, etc.).




## 10. TRAFFIC FLOW (EXAMPLE)




1. User sends request to your application.
2. Traffic goes through Load Balancer.
3. Ingress routes based on host/path.
4. Request goes to Service (ClusterIP).
5. Service load balances traffic to healthy Pods.

**EXAMPLE FLOW**

https://app.example.com/api → Ingress → Service (ClusterIP) → Pods
Pods process the request and return the response back to the user.

## TRAFFIC FLOW INSIDE CLUSTER




POD A uses DNS to find the service.
Traffic is load balanced to one of the backend Pods.

## KEY TAKEAWAY

Kubernetes networking provides a reliable, secure, and scalable way to connect applications.
Understand Services, Ingress, DNS, and Policies to build production-ready systems.

VERIQTA
ENGINEERING EXCELLENCE



# 16. KUBERNETES CONFIGURATION AND STORAGE

Manage application configuration and data reliably in Kubernetes.

## 1 CONFIGMAPS

Store non-sensitive configuration data as key-value pairs.

YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: myapp
  APP_ENV: production
  LOG_LEVEL: info
```

**USE IN POD**

```yaml
envFrom:
- configMapRef:
    name: app-config
```

## 2 SECRETS

Store sensitive information securely (base64 encoded).

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=
  password: cGFzc3dvcmQ=
```

**USE IN POD**

```yaml
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: username
```

## 3 VOLUMES

Provide storage that is accessible to containers in a Pod.

- emptyDir – Temporary storage
- configMap – Mount ConfigMap
- secret – Mount Secret
- hostPath – Node filesystem
- persistentVolumeClaim – Use PVC

**EXAMPLE: CONFIGMAP VOLUME**

```yaml
volumes:
- name: app-config
  configMap:
    name: app-config
containers:
- name: app
  image: myapp:1.0
  volumeMounts:
  - name: app-config
    mountPath: /etc/config
```

## 4 PERSISTENTVOLUMES (PV)

Cluster resources that represent physical storage in the cluster.

- Created by admin or dynamically via StorageClass
- PV has a lifecycle independent of a Pod
- Reclaim Policies: Retain | Delete | Recycle

**EXAMPLE PV**

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-data
spec:
  capacity:
    storage: 10Gi
  accessModes:
  - ReadWriteOnce
persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /data/k8s
```

## 5 PERSISTENTVOLUMECLAIMS (PVC)

Requests storage by users. Binds to a PV or gets provisioned dynamically.

- Created by users in a namespace
- Binds to a matching PV
- Abstracts storage details from Pods
- Supports access modes

**EXAMPLE PVC**

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-data
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: fast-storage
```

## 6 STORAGECLASSES

Define how storage should be provisioned (manually or dynamically).

- Specify provisioner
- Parameters for storage
- Supports different backends
- Used by PVC for dynamic provisioning

**EXAMPLE STORAGECLASS**

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

## 7 DYNAMIC PROVISIONING

Automatically creates PVs when a PVC is created.

PVC (User Request) → STORAGECLASS (Defines Storage) → PROVISIONER (Creates PV) → PV (Bound to PVC)

- No manual PV creation
- Uses StorageClass
- Scalable and efficient
- Works with cloud storage

## 8 STATEFUL WORKLOADS

Applications that require stable storage and network identity.

- Use StatefulSet
- Stable Pod identity
- Persistent storage per replica
- Ordered deployment
- Ordered scaling
- Ideal for databases, queues, and stateful apps

## 9 CONFIGURATION MANAGEMENT

Best practices for managing configuration.

- Use ConfigMaps for non-sensitive data
- Use Secrets for sensitive data
- Keep configuration separate from image
- Use environment variables and volumes
- Version control your configs
- Automate updates and rollouts
- Validate configuration at startup

## HOW IT ALL WORKS TOGETHER

CONFIGURATION (ConfigMaps, Secrets) → POD (Uses Config and Secrets) → PVC (Requests Storage) → STORAGECLASS (Defines How to Provision) → PV (Storage Provided) → POD MOUNTS (Storage and Runs)

## KEY TAKEAWAYS

- Use ConfigMaps for non-sensitive configuration.
- Use Secrets for sensitive information.
- Use PVCs to request storage.
- Use StorageClasses for dynamic provisioning.
- Stateful workloads need persistent storage.
- Separate configuration from container images.

## BEST PRACTICES

- Follow least privilege for Secrets.
- Backup persistent data regularly.
- Monitor storage usage and performance.
- Test disaster recovery for stateful apps.

VERIQTA
ENGINEERING EXCELLENCE



# 17. PRODUCTION KUBERNETES OPERATIONS

Run reliable, scalable, and resilient workloads in production.

## 1 REQUEST' AND LIMITS

Guarantee resources and prevent noisy neighbor problems.

- Requests = Minimum resources guaranteed
- Limits = Maximum resources allowed

**EXAMPLE**

```yaml
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "1Gi"
```

## 2 LIVENESS PROBES

Detects when a container is unhealthy and should be restarted.

**EXAMPLE (HTTP)**

```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```

If probe fails, Kubernetes restarts the container.

## 3 READINESS PROBES

Detects when a container is ready to receive traffic.

**EXAMPLE (HTTP)**

```yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```

If probe fails, Pod is removed from Service.

## 4 STARTUP PROBES

Handles slow-starting containers. Disables liveness/readiness until startup succeeds.

**EXAMPLE (HTTP)**

```yaml
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
  timeoutSeconds: 2
```

Use for apps with long initialization time.

## 5 HORIZONTAL POD AUTOSCALER

Automatically scales Pods based on CPU or custom metrics.

**EXAMPLE**

```yaml
apiVersion: autoscaling/v2
kind: HoricontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Scales up on demand. Scales down when not needed.

## 6 POD DISRUPTION BUDGETS

Ensures minimum Pods stay available during disruptions.

**EXAMPLE**

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web-app
```

Prevents voluntary disruptions from taking down your app.

## 7 NODE AFFINITY

Schedule Pods on preferred or required nodes.

**EXAMPLE (REQUIRED)**

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnored
    DuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: node-type
          operator: In
          values:
          - production
```

Ensures Pods run on the right nodes.

## 8 TAINTS AND TOLERATIONS

Prevent Pods from scheduling on certain nodes unless tolerated.

**TAINT NODE**

```
kubectl taint nodes node1
key=value:NoSchedule
```

**TOLERATION IN POD**

```yaml
tolerations:
- key: "key"
  operator: "Equal"
  value: "value"
  effect: "NoSchedule"
```

Protect dedicated nodes (e.g., for system workloads).

## 9 RBAC (ROLE-BASED ACCESS CONTROL)

Control who can do what in the cluster.

**ROLE EXAMPLE**

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

**ROLEBINDING EXAMPLE**

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Apply least privilege. Grant only what is necessary.

## 10 PRODUCTION RELIABILITY

Build resilient, highly available, and observable systems.

- Use resource requests and limits
- Implement all probes (startup, liveness, readiness)
- Autoscale based on real metrics
- Ensure high availablity across zones
- Protect workloads with PDBs
- Use affinity and taints for proper scheduling
- Secure access with RBAC
- Monitor, alert, and respond fast
- Test failure scenarios regularly
- Document and automate everything

## PRODUCTION OPERATIONS CHECKLIST

- Set resource requests and limits
- Configure liveness probe
- Configure readiness probe
- Add startup probe if needed
- Enable HPA
- Configure PDB
- Use node affinity where needed
- Apply taints and tolerations
- Secure with RBAC
- Monitor and alert
- Backup application data
- Test recovery and failover

VERIQTA
ENGINEERING EXCELLENCE



# 18. CI/CD FUNDAMENTALS

**AUTOMATE. VALIDATE. DELIVER. REPEAT.**

Deliver valuable software faster, safer, and more reliably.

## 1 CONTINUOUS INTEGRATION (CI)

Developers integrate code frequently into a shared repository.

- Automated build on every commit
- Early detection of issues
- Improved code quality
- Fast feedback to developers

## 2 CONTINUOUS DELIVERY (CD)

Code is always in a deployable state and ready for release.

- Automated testing and quality gates
- Manual approval for production
- Consistent and repeatable releases
- Reduced risk and faster delivery

## 3 CONTINUOUS DEPLOYMENT (CD)

Every change that passes all checks is automatically deployed to production.

- Fully automated release to production
- Rapid value delivery
- Lower lead time and human error
- Requires strong automation and tests

## 4 PIPELINE STAGES

### 1 BUILD
Compile source code and create artifacts.

**Example:**
mvn clean package
docker build

### 2 TEST
Run automated tests to ensure quality.

**Example:**
unit tests
integration tests

### 3 PACKAGE
Package the application for distribution.

**Example:**
create JAR, WAR or Docker image

### 4 SCAN
Scan for security, vulnerabilities, and compliance issues.

**Example:**
SAST, SCA, Container Scan, Secret Scan

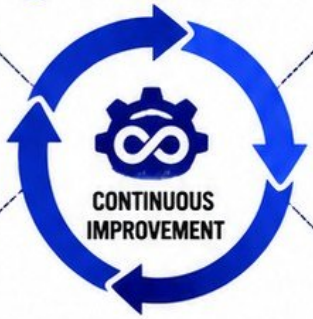
### 5 DEPLOY
Deploy to the target environment.

**Example:**
Kubernetes, VM, Serverless

### 6 VALIDATE
Verify the deployment and application health.

**Example:**
Smoke tests, Health checks, Monitoring

## 5 FEEDBACK LOOPS




**MONITOR**
Collect metrics, logs, and traces from production.

**FEEDBACK**
Share insights with the team quickly.

**LEARN**
Analyze data, find root causes, and improve processes.

**IMPROVE**
Enhance code, tests, infrastructure, and automation.

## 6 WHY CI/CD MATTERS

- Faster delivery of new features
- Higher code quality
- Early detection of bugs and vulnerabilities
- Consistent and repeatable releases
- Better collaboration and visibility
- Lower risk and faster recovery

## 7 KEY PRINCIPLES

- EVERY CHANGE IS BUILT
- EVERY BUILD IS TESTED
- EVERY RELEASE IS SAFE
- EVERY CHANGE ADDS VALUE
- EVERY OUTCOME IS FEEDBACK

## 8 EXAMPLE END-TO-END CI/CD PIPELINE FLOW

DEVELOPER PUSH CODE → CI SERVER TRIGGERS PIPELINE → BUILD COMPILE CODE → TEST RUN TESTS → PACKAGE CREATE ARTIFACT → SCAN SECURITY SCAN → DEPLOY TO STAGING/PROD → VALIDATE VERIFY & MONITOR → FEEDBACK LOOP

## BEST PRACTICES

- Commit small and often.
- Automate everything.
- Test early and test often.
- Monitor and get feedback.

## AVOID

- Manual and repetitive tasks.
- Skipping tests or quality gates.
- Deploying unverified code.
- Ignoring feedback and metrics.

# 19. BUILDING PRODUCTION CI PIPELINES

**BUILD RELIABLE. AUTOMATE QUALITY. SHIP CONFIDENTLY.**

A production-ready pipeline delivers fast, secure, and high-quality software.

## CI PIPELINE STAGES

### 1 SOURCE CHECKOUT

- Checkout code from repository
- Fetch latest changes
- Validate commit and branch

**EXAMPLE**

```
git clone <repo>
git checkout <branch>
```

### 2 DEPENDENCY INSTALLATION

- Install required dependencies
- Use lock files for consistency
- Cache dependencies to speed up builds

**EXAMPLE**

```
npm ci
pip install -r
requirements.txt
```

### 3 COMPILATION

- Compile source code
- Fail fast on compilation errors
- Generate build artifacts

**EXAMPLE**

```
mvn clean package
gradle build
go build ./...
```

### 4 UNIT TESTING

- Run unit tests
- Validate individual components
- Genure high code coverage

**EXAMPLE**

```
mvn test
pytest
go test ./...
```

### 5 INTEGRATION TESTING

- Test component interactions
- Validate APIs, services, and DB
- Run in isolated test environment

**EXAMPLE**

```
mvn verify
pytest -m integration
newman run tests/
```

### 6 CODE QUALITY

- Run linters and formatters
- Check code smells and complexity
- Enforce quality gates

**EXAMPLE**

```
sonar-scanner
eslint .
flake8
```

### 7 SECURITY SCANNING

- Scan dependencies for vulnerabilities
- Scan code for secrets and risks
- Fail on high or critical issues

**EXAMPLE**

```
trivy fs .
snyk test
gitleaks detect
```

### 8 ARTIFACT CREATION

- Package build output
- Version and tag artifacts
- Publish to artifact repository

**EXAMPLE**

```
docker build -t app:1.0
docker push registry/
app:1.0
```

### 9 PIPELINE FAILURE HANDLING

- Fail fast on errors
- Clear and helpful error messages
- Notify the team
- Stop unsafe deployments
- Allow easy re-runs

**EXAMPLE**

```
notify: slack/email
archive logs
rollback if needed
```

## PRODUCTION PIPELINE BEST PRACTICES

- KEEP PIPELINES FAST AND RELIABLE
- FAIL EARLY AND FAIL FAST
- AUTOMATE EVERYTHING REPEATABLE
- KEEP ENVIRONMENTS CONSISTENT
- USE PARALLEL EXECUTION WHERE POSSIBLE
- STORE SECRETS SECURELY
- VERSION AND IMMUTABLE ARTIFACTS
- MONITOR PIPELINE PERFORMANCE
- REVIEW AND IMPROVE CONTINUOUSLY
- SECURE THE PIPELINE ITSELF

## PIPELINE QUALITY GATES

- ALL UNIT TESTS MUST PASS
- NO CRITICAL SECURITY VULNERABILITIES
- CODE COVERAGE MEETS THRESHOLD
- CODE QUALITY GATE PASSES
- BUILD AND TESTS COMPLETE SUCCESSFULLY
- ARTIFACTS CREATED AND VERIFIED

## COMMON FAILURE SCENARIOS

- **COMPILATION FAILURE**
  Code does not compile or build fails.
- **TEST FAILURE**
  Unit or integration tests fail.
- **QUALITY GATE FAILURE**
  Code quality does not meet standards.
- **SECURITY SCAN FAILURE**
  Vulnerabilities or secrets detected.
- **INFRASTRUCTURE FAILURE**
  Services, databases, or tools unavailable.
- **TIMEOUT FAILURE**
  Job exceeds maximum allowed time.
- **ARTIFACT PUBLISH FAILURE**
  Unable to publish artifact to repository.

## EXAMPLE END-TO-END CI PIPELINE FLOW

CODE COMMIT → SOURCE CHECKOUT → DEPENDENCY INSTALL → COMPILE → UNIT TESTS → INTEGRATION TESTS → CODE QUALITY → SECURITY SCAN → CREATE ARTIFACT → DEPLOY → VALIDATE → FEEDBACK LOOP

## REMEMBER

- AUTOMATE. DON'T MANUAL-REPEAT.
- TEST. DON'T ASSUME.
- MEASURE. DON'T GUESS.
- SECURE. DON'T EXPOSE.
- DELIVER. DON'T DELAY.
- IMPROVE. DON'T STOP.

## THE GOAL

Deliver high-quality software to production quickly, safely, and reliably.
Every time.

VERIQTA
ENGINEERING EXCELLENCE



# 20. ARTIFACT AND CONTAINER MANAGEMENT

**MANAGE, SECURE, AND DELIVER ARTIFACTS WITH CONFIDENCE.**

The foundation of reliable, repeatable, and secure software delivery.

## 1 BUILD ARTIFACTS

Outputs of the build process ready for use or deployment.

- Compiled binaries
- Libraries and packages
- Configuration files
- Documentation
- Test reports
- Build metadata

**EXAMPLE**

```
app.jar
dist/app.zip
package.whl
build-report.html
```

## 2 ARTIFACT REPOSITORIES

Store and manage build artifacts securely.

- Centralized storage
- Versioned artifacts
- Access control
- Metadata and search
- High availability
- Integration with CI/CD

**EXAMPLE**

JFrog Artifactory
Sonatype Nexus
AWS CodeArtifact
Azure Artifacts

## 3 CONTAINER REGISTRIES

Store and distribute container images.

- OCI compliant
- Secure authentication
- Image scanning
- Replication
- High availability
- Geo redundancy

**EXAMPLE**

Docker Hub
Amazon ECR
Google Artifact Registry
Azure Container Registry

## 4 VERSIONING

Track and manage versions consistently across artifacts and images.

v1.2.3

- Semantic Versioning (SemVer)
- Consistent version scheme
- Traceability
- Reproducibility
- Rollback capability
- Clear release history

**EXAMPLE**

1.0.0
1.0.1
1.1.0
2.0.0

## 5 IMMUTABLE ARTIFACTS

Once published, artifacts and images must never change.

- Prevent tampering
- Ensure consistency
- Enable safe promotion
- Improve security
- Support rollback
- Build trust

**EXAMPLE**

```
app-1.0.0.jar
app-1.0.0.jar (never overwrite)
myapp:1.0.0 (immutable tag)
myapp@sha256:abcd...
```

## 6 IMAGE TAGGING

Use clear and meaningful tags for container images.

- Use SemVer or date tags
- Avoid "latest" in production
- Use commit SHA for traceability
- Multi-tag strategy
- Environment specific tags
- Clear and consistent naming

**EXAMPLE**

```
myapp:1.2.0
myapp:1.2.0-20240510
myapp:1.2.0-abc1234
myapp:stable (non-prod only)
```

## 7 PROMOTION

Move artifacts or images through environments safely.

- Dev → Test → Staging → Prod
- No rebuilding in higher envs
- Automated promotion
- Approval gates
- Traceable promotions
- Consistent deployments

**EXAMPLE**

Promote myapp:1.2.0
from Dev → Test → Staging → Prod
using the same immutable image.

## 8 RETENTION

Retain artifacts and images based on policies.

- Define retention policies
- Keep important versions
- Clean up old artifacts
- Reduce storage cost
- Comply with regulations
- Balance cost and availability

**EXAMPLE**

Keep last 10 releases
Keep for 30 days in non-prod
Keep for 1 year in production

## 9 PROVENANCE

Know where artifacts and images come from.

- Build metadata
- Source repository
- Commit SHA
- Builder information
- Build environment
- Reproducible builds

**EXAMPLE**

Built from: git@github.com/org/app
Commit: abc1234
Built by: CI-Pipeline-01
At: 2024-05-10T12:30:00Z

## 10 SUPPLY CHAIN INTEGRITY

Ensure software is trusted from source to production.

- Verify dependencies
- Scan for vulnerabilities
- Sign artifacts and images
- Use trusted sources
- SBOM (Software Bill of Materials)
- End-to-end traceability

**EXAMPLE**

Signed image
Verified dependencies
SBOM attached
Policy enforcement

## TYPICAL ARTIFACT & CONTAINER LIFECYCLE

1 **BUILD** → Code is compiled and artifacts are created.

2 **STORE** → Artifacts/images stored in secure repositories.

3 **VERSION** → Versioned and tagged for traceability.

4 **SCAN** → Scanned for security vulnerabilities and compliance.

**PROMOTE** → Promoted through environments with approval gates.

**DEPLOY** → Deployed to target environment consistently.

7 **MONITOR** → Monitor, validate and gather feedback.

8 **RETAIN / CLEAN** → Retain per policy and clean up old versions.

## BEST PRACTICES

- Always use immutable artifacts and images.
- Never rebuild in higher environments.
- Use semantic versioning consistently.
- Scan early and scan often.
- Sign artifacts and images.
- Maintain provenance and traceability.
- Enforce retention and cleanup policies.
- Automate promotion with approvals.

## COMMON ANTI-PATTERNS

- Using "latest" tags in production.
- Overwriting artifacts or images.
- Skipping vulnerability scans.
- Manual promotion without approvals.
- Keeping too many unused versions.
- No visibility into provenance.
- No artifact retention policy.
- Ignoring supply chain security.

VERIQTA
ENGINEERING EXCELLENCE



# 21. PRODUCTION DEPLOYMENT STRATEGIES

**DELIVER NEW VALUE TO USERS SAFELY, QUICKLY, AND RELIABLY.**
Choose the right strategy for your application and risk profile.

## 1 RECREATE

Stops all old versions before starting new version.

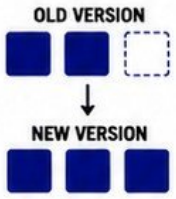



- Simple and easy
- Downtime occurs
- Not suitable for critical apps

**USE WHEN:** Downtime is acceptable.

## 2 ROLLING DEPLOYMENT

Gradually replaces old instances with new ones.

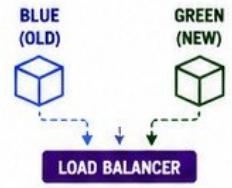



- Zero/low downtime
- Maintains availability
- Default in Kubernetes

**USE WHEN:** You need high availability.

## 3 BLUE-GREEN

Run two identical environments. Switch traffic to the new one.

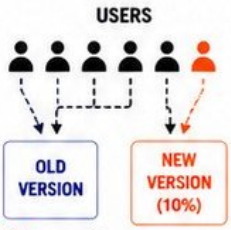



- Zero downtime
- Easy rollback (switch back)
- Requires 2x infrastructure

**USE WHEN:** You want fast rollback and stability.

## 4 CANARY

Deploy to a small subset of users first.




- Lower risk
- Real user validation
- Slow rollout

**USE WHEN:** You want to validate with real traffic.

## 5 PROGRESSIVE DELIVERY

Automate incremental rollout to higher % of traffic.




- Automated progression
- Monitors metrics
- Auto rollback on failure

**USE WHEN:** You want safe, automated rollouts.

## 6 FEATURE FLAGS

Release features without releasing code.

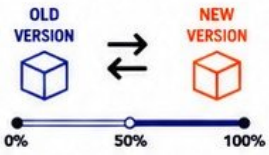


- Enable/disable features
- Instant control
- Decouple release from deploy

**USE WHEN:** You want to control feature exposure.

## 7 TRAFFIC SHIFTING

Dynamically shift traffic between environments or versions.




- Fine-grained control
- Supports canary & progressive
- Use with service mesh / LB

**USE WHEN:** You need precise traffic control.

## 8 ZERO-DOWNTIME DEPLOYMENTS

Ensure continuous availability during deployments.




- No user disruption
- Requires health checks
- Proper readiness & probes

**USE WHEN:** Your application must always be available.

## 9 VALIDATION

Verify the new version before full rollout or completion.



- Automated tests
- Health & smoke checks
- Business metric validation

**USE WHEN:** You need confidence before full release.

## 10 ROLLBACK

Quickly revert to a previous stable version.



- Fast recovery
- Minimize impact
- Always test rollback

**USE WHEN:** Issues are detected after deployment.

## STRATEGY COMPARISON

| STRATEGY | DOWNTIME | RISK LEVEL | INFRA REQUIREMENT | DEPLOY SPEED | ROLLBACK SPEED | BEST FOR |
|---|---|---|---|---|---|---|
| RECREATE | HIGH | HIGH | LOW | FAST | SLOW | NON-CRITICAL APPS |
| ROLLING DEPLOYMENT | LOW | MEDIUM | LOW | MEDIUM | MEDIUM | MOST APPLICATIONS |
| BLUE-GREEN | NONE | LOW | HIGH (2X) | FAST | FAST | CRITICAL APPLICATIONS |
| CANARY | LOW | LOW | LOW | SLOW | FAST | RISKY CHANGES |
| PROGRESSIVE DELIVERY | LOW | LOW | LOW | SLOW | FAST | CONTINUOUS DELIVERY |
| FEATURE FLAGS | NONE | LOW | LOW | FAST | N/A | FEATURE ROLLOUTS |
| TRAFFIC SHIFTING | LOW | LOW | LOW | MEDIUM | FAST | GRANULAR CONTROL |

## DEPLOYMENT SUCCESS CHECKLIST

- TESTED IN STAGING
- AUTOMATED TESTS PASS
- HEALTH CHECKS CONFIGURED
- MONITORING & ALERTS IN PLACE
- ROLLBACK PLAN VALIDATED
- STAKEHOLDERS NOTIFIED
- POST-DEPLOYMENT VALIDATION

## GOLDEN RULES

- Always deploy small and verify.
- Observe, then increase.
- Automate everything.
- Monitor continuously.
- Have a rollback plan.
- Deliver value safely.

## COMMON PITFALLS TO AVOID

- Skipping tests and validation.
- No rollback plan.
- Deploying too fast.
- Ignoring metrics and alerts.
- No canary or staged rollout.
- Hardcoding configurations.

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 22. GITOPS AND CONTINUOUS DELIVERY

Automate delivery. Ensure consistency. Operate with confidence.

## 1. GIT AS SOURCE OF TRUTH

- All configurations in Git
- Single source of truth
- Versioned and auditable
- Collaborative and reviewable
- Enables traceability

## 2. DESIRED STATE

- Define the desired state in Git
- Cluster converges to that state
- No manual changes
- Declarative and predictable

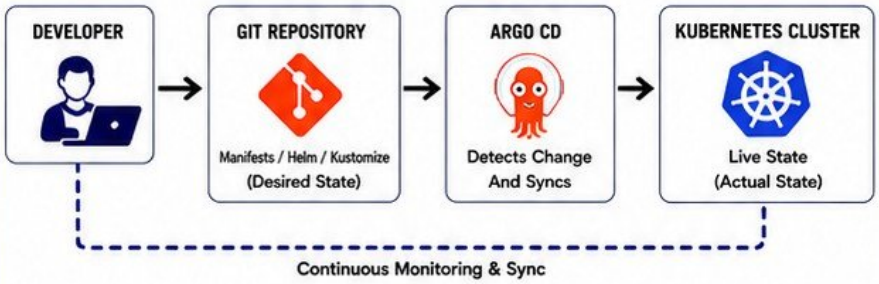
## 3. GITOPS WORKFLOW

1. Developer changes code
2. Push to Git repository
3. GitOps tool detects change
4. Sync desired state to cluster
5. Cluster becomes consistent
6. Continuous monitoring

## 4. ARGO CD

- Declarative CD tool
- Monitors Git repositories
- Syncs to Kubernetes clusters
- Web UI and CLI
- Built-in observability

## 5. APPLICATION SYNCHRONIZATION




## 6. DRIFT DETECTION

| NO DRIFT | DRIFT DETECTED |
|---|---|
| Actual State = Desired State | Actual State ≠ Desired State |
| No action needed | Trigger sync to fix |

GitOps continuously detects and highlights drift.

## 7. AUTOMATED SYNCHRONIZATION

- Automatic sync keeps cluster in desired state
- Self-healing ensures consistency
- Configurable sync policies
- Manual approval when needed
- Safe, reliable and repeatable

## 8. ROLLBACKS

- Revert Git commit
- GitOps tool detects change
- Automatically rolls back
- Quick recovery to last known good state
- Safe and auditable rollbacks

## 9. ENVIRONMENT PROMOTION

- Promote changes across environments
- Dev → Test → Staging → Prod
- Separate configs per environment
- Controlled and sequential promotion
- Traceable and consistent releases

## GITOPS REPOSITORY STRUCTURE EXAMPLE

```
apps/
└─ my-app/                          Base manifests
   ├─ base/
   │  ├─ deployment.yaml            Environment overlays
   │  ├─ service.yaml
   │  └─ configmap.yaml
   └─ overlays/
      ├─ dev/                       kustomization.yaml
      │                             values.yaml
      ├─ staging/                   kustomization.yaml
      │                             values.yaml
      └─ prod/                      kustomization.yaml
                                    values.yaml
```

## ARGO CD SYNC POLICIES

| | |
|---|---|
| MANUAL | Sync only when manually triggered. |
| AUTOMATIC | Sync automatically on changes. |
| AUTOMATIC + PRUNE | Sync and remove resources no longer defined in Git. |
| AUTOMATIC + SELF HEAL | Sync and revert any drift automatically. |

## EXAMPLE: APPLICATION SYNC STATUS

```
$ argocd app get my-app
Name:          my-app
Project:       default
Server:        https://kubernetes.default.svc
Namespace:     my-app
Status:        ● Healthy    ✓ Synced
Revision:      a1b2c3d
Updated:       2 minutes ago
Sync Policy:   Automatic
Health:        Healthy
Message:       Successfully synced

$ argocd app diff my-app
No differences found
```

## GITOPS BEST PRACTICES

- Keep Git repository clean and organized
- Use meaningful commit messages
- Review changes before merging
- Automate tests before promotion
- Monitor and alert on drift and failures
- Document and follow GitOps standards

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 23. OBSERVABILITY FUNDAMENTALS

See Everything. Understand Deeply. Act Quickly.

## 1. MONITORING VS OBSERVABILITY

| MONITORING | | OBSERVABILITY |
|---|---|---|
| Tells you WHAT is happening | | Tells you WHY it is happening |
| Predefined dashboards and alerts | VS | Exploratory and flexible |
| Reactive | | Proactive |
| Based on known unknowns | | Helps discover unknown unknowns |
| Answers known questions | | Answers any question |

KEY: Monitoring alerts you. Observability empowers you to understand and solve the root cause.

## 2. THE FIVE PILLARS OF OBSERVABILITY

| | |
|---|---|
| METRICS | Numeric measurements over time. Aggregated, fast, and efficient. Example: CPU usage, request rate, latency. |
| LOGS | Discrete events with timestamps. Detailed records for deep investigation. Example: Error logs, access logs. |
| TRACES | Request flow across services. Shows latency and dependencies. Example: Distributed request trace. |
| EVENTS | Important state changes or notifications. Lightweight and meaningful. Example: Deployment succeeded. |
| TELEMETRY | The signals collected from systems. Includes metrics, logs, traces, and events. |

## 3. GOLDEN SIGNALS

- **LATENCY** — How long does it take to serve a request?
- **TRAFFIC** — How much demand is being placed?
- **ERRORS** — How many requests are failing?
- **SATURATION** — How full is your system?

Golden signals help you understand the health of any service.

## 4. RED METHOD

Focuses on the behavior of your service from the user's perspective.

| | |
|---|---|
| R | **RATE** — Requests per second |
| E | **ERRORS** — Error rate (4xx, 5xx) |
| D | **DURATION** — Request duration (latency) |

Best for understanding service reliability and performance.

## 5. USE METHOD

Focuses on the resources of your system from the infrastructure perspective.

| | |
|---|---|
| U | **UTILIZATION** — How busy resources are |
| S | **SATURATION** — How full resources are |
| E | **ERRORS** — Error rate of the resources |

Best for understanding infrastructure health and capacity.

## 6. OBSERVABILITY ARCHITECTURE

| SOURCES | COLLECTION | PROCESSING & STORAGE | VISUALIZATION & ANALYSIS | ACTION & RESPONSE |
|---|---|---|---|---|
| Applications | Metrics Collectors | Metrics DB (Time Series) | Dashboards | Alerting |
| Services | Log Collectors | Log Storage (Indexed) | Queries | Incident Response |
| Infrastructure | Trace Collectors | Trace Storage (Raw / Indexed) | Alerts | Automation |
| Databases | Event Collectors | Event Store (Optional) | Reports | Notifications |
| Kubernetes | Exporters / Agents | | Exploration | Runbooks |

FEEDBACK LOOP: Learn, Improve, and Iterate

## BEST PRACTICES

- Instrument everything
- Use meaningful metrics
- Correlate logs, metrics, and traces
- Set smart alerts
- Reduce noise and false alerts
- Continuously improve visibility

PAGE 24

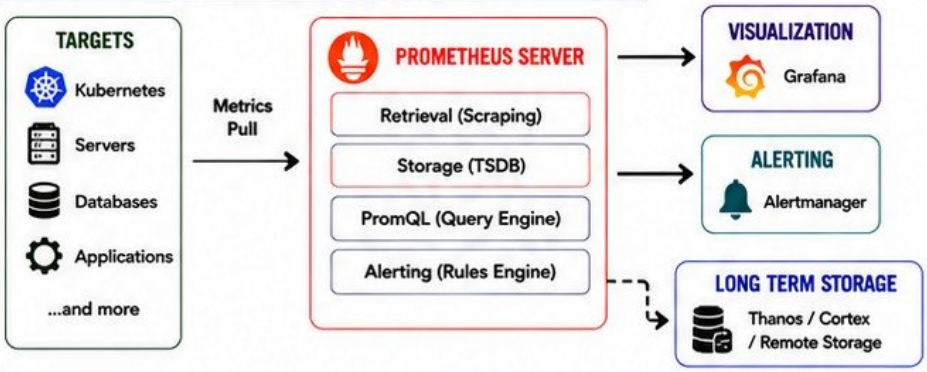
VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 24. PROMETHEUS AND METRICS

Collect. Store. Query. Alert. Understand your systems.

## 1. PROMETHEUS ARCHITECTURE




## 2. TARGETS

- Anything that exposes metrics via HTTP endpoint
- Defined in scrape configurations
- Static or service discovered
- Health and status are monitored
- Examples: nodes, pods, services, databases

**EXAMPLE TARGET**

```
http://node1:9100/metrics
```

## 3. EXPORTERS

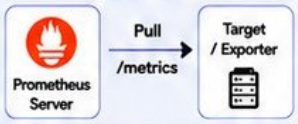
- Exporters expose metrics in Prometheus format
- Each component has its own exporter
- Run alongside the target
- Examples:

| COMPONENT | EXPORTER |
|---|---|
| Linux / Servers | Node Exporter |
| Kubernetes | kube-state-metrics |
| Databases | mysqld_exporter |
| HTTP Services | blackbox_exporter |
| Applications | Custom Exporter |

## 4. SCRAPING

- Prometheus pulls metrics at regular intervals (scrape interval)
- Default: every 15 seconds
- Configurable per job
- Failed scrapes are retried

**SCRAPE FLOW**




## 5. METRICS

- Metrics are time series data
- Consist of:

**METRIC_NAME{LABELS} VALUE TIMESTAMP**

Example:

```
http_requests_total{method="GET", status="200", job="api"} 1027 1715338812
```

## 6. LABELS

- Key-value pairs that add context to metrics
- Used for filtering, grouping, and aggregation
- Examples: job, instance, endpoint, method, status, namespace, pod

**EXAMPLE**

```
http_requests_total{
job="api",
instance="10.0.0.5:8080",
method="POST",
status="500"
}
```

## 7. PROMQL

- Powerful query language
- Filter, aggregate, and analyze metrics
- Used in Grafana dashboards and alerting rules

**EXAMPLE QUERIES**

```
# Total requests per second
rate(http_requests_total[5m])

# Error rate (5xx)
sum(rate(http_requests_total{status=~"5.."}[5m]))

# CPU usage by instance
100 - (avg by (instance) (rate(node_cpu_seconds_total{
mode="idle"}[5m])) * 100)
```

## 8. RECORDING RULES

- Pre-calculate and store query results
- Improve performance
- Keep dashboards and alerts fast and reliable

**EXAMPLE**

```
groups:
- name: api.rules
  rules:
  - record: job:http_requests:rate5m
    expr: rate(http_requests_total[5m])
```

## 9. ALERTING RULES

- Define conditions when alerts should fire
- Integrated with Alertmanager
- Support for severity, annotations, and routing

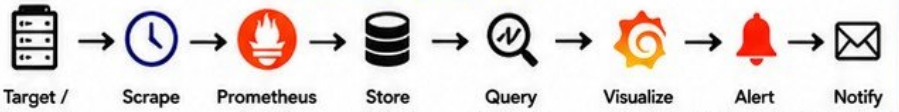
**EXAMPLE**

```
groups:
- name: api.alerts
  rules:
  - alert: HighErrorRate
    expr: rate(http_requests_total{
      status=~"5.."}[5m]) > 0.05
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: High error rate detected
```

## 10. PRODUCTION MONITORING

- Use meaningful metrics (RED/USE)
- Set good label practices
- Avoid high cardinality labels
- Create service dashboards
- Alert on symptoms, not metrics in isolation
- Use recording rules for heavy queries
- Implement alert routing and escalation
- Test alerts regularly
- Monitor Prometheus itself
- Plan for retention and storage

## PROMETHEUS DATA FLOW




## KEY TAKEAWAY

Prometheus gives you deep visibility into your systems. With the right metrics, queries, and alerts, you can detect issues before users do.

**Measure. Alert. Improve.**


VERIQTA SOCIAL
YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

PAGE 25

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 25. GRAFANA AND PRODUCTION DASHBOARDS

Visualize everything. Detect issues fast. Drive better decisions.

## 1. DATA SOURCES

- Connect to your data
- Multiple data sources
- Real-time querying
- Secure and scalable
- Examples:
  - Prometheus
  - Loki
  - Elasticsearch
  - MySQL / PostgreSQL
  - CloudWatch
  - And more

## 2. DASHBOARDS

- Collection of panels
- Your system overview
- Share and collaborate
- Folder organization
- Import / export
- Version control (Git)
- Provision dashboards

One dashboard = One purpose.

## 3. PANELS

- Visualize your metrics
- Many panel types
- Flexible visualization
- Drill down into details
- Common panel types:
  - Time series
  - Stat
  - Gauge
  - Bar gauge
  - Heatmap
  - Table
  - Pie chart
  - Logs

## 4. VARIABLES

- Make dashboards dynamic
- Reusable and flexible
- Filter by environment, service, instance, etc.
- Variable types:
  - Query variable
  - Custom variable
  - Interval variable
  - Constant variable
  - Ad hoc filters

## 5. ALERTING

- Alert on important events
- Integrated with Alertmanager
- Multi-channel notifications
- Silence and scheduling
- Alert rules from panels
- Example channels:
  - Email
  - Slack
  - PagerDuty
  - Telegram
  - Webhook

## 6. INFRASTRUCTURE DASHBOARDS

Monitor your servers, networks, and cloud infrastructure.

- CPU, Memory, Disk, Network
- Load, Uptime, Processes
- Node Exporter metrics
- Cloud resources




## 7. APPLICATION DASHBOARDS

Monitor application health and performance.

- Request rate, latency, errors
- Apdex, throughput
- Business metrics
- Custom application metrics




## 8. KUBERNETES DASHBOARDS

Monitor Kubernetes clusters and workloads.

- Cluster health
- Pods, Nodes, Deployments
- CPU, Memory, Disk usage
- Kube-state-metrics




## 9. DASHBOARD DESIGN

- Start with a clear goal
- Use meaningful titles
- Keep it simple and clean
- Use consistent layout
- Use colors intentionally
- Show trends, not tables
- Group related panels
- Add annotations

**GOOD DASHBOARD DESIGN**

- ✓ Clear
- ✓ Focused
- ✓ Actionable
- ✓ Consistent
- ✓ Easy to read

## 10. AVOIDING DASHBOARD OVERLOAD

- Too many panels
- Too many metrics
- Irrelevant data
- No clear purpose
- Complex and cluttered views
- Overlapping information
- Lack of drill-down

BAD EXAMPLE



Show less. Understand more.
Focus on what matters.

## 11. GRAFANA BEST PRACTICES

- Use templates and variables
- Use consistent naming
- Set default time ranges
- Use annotations for events
- Use recording rules for heavy queries
- Optimize queries
- Test alerts regularly
- Document your dashboards
- Keep dashboards in Git
- Review and improve continuously

A great dashboard saves time, prevents incidents, and builds trust.

## 12. GRAFANA WORKFLOW

CONNECT DATA SOURCE → BUILD DASHBOARD → VISUALIZE INSIGHTS → ALERT & TAKE ACTION

**FROM DATA TO ACTIONABLE INSIGHTS**

Collect → Visualize → Alert → Act → Improve

- Observe your system
- Detect issues early
- Respond quickly
- Continuously improve

**REMEMBER:**

Dashboards are not for datas. They are for decisions.

Right data. Right view.

Right alert. Right action.

Better systems. Happier users. Stronger teams.

VERIQTA SOCIAL

YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

PAGE 26

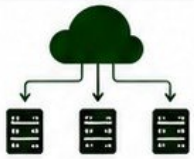
VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
FROM ENGINEERING FOUNDATIONS TO PRODUCTION DEVOPS

# 26. LOGGING AND DISTRIBUTED TRACING

Collect Logs. Trace Requests. Find Root Cause Faster.

## 1. CENTRALIZED LOGGING



- Collect logs from all systems in one place
- Eliminate log silos
- Search, analyze, and act fast
- Improve visibility and operational efficiency

**TOOLS**
Elasticsearch, Loki, CloudWatch, Splunk

## 2. STRUCTURED LOGS

- Logs in key-value or JSON format
- Easy to parse and query
- Consistent fields
- Better filtering and analysis
- Essential for automation

**JSON LOG EXAMPLE**

```
{
"timestamp": "2025-05-21T2:30:002",
"level": "ERROR",
"service": "checkout",
"message": "Payment failed",
"order_id": "A12345",
"user_id": "U67890",
"status": 500
}
```

## 3. LOG AGGREGATION

- Aggregate logs from multiple sources
- Index and store efficiently
- Enable fast search
- Retention and archiving
- Real-time analysis

**LOG AGGREGATION FLOW**

App Logs, Server Logs, K8s Logs, DB Logs → Log Aggregator (Fluent Bit / Filebeat) → Log Store (Elasticsearch / Loki / CloudWatch)

## 4. CORRELATION IDS

- Unique ID per request
- Track a request across services and systems
- Include in logs, headers, and traces
- Essential for debugging and root cause analysis

**EXAMPLE**

Correlation ID:
`a1b2c3d4e5f67890`

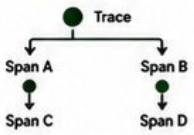
Header:
`X-Correlation-ID: a1b2c3d4e5f67890`

## 5. DISTRIBUTED TRACING

- Trace requests across microservices
- End-to-end visibility
- Identify latency and bottlenecks
- Understand service dependencies

**TOOLS**
Jaeger, Zipkin, AWS X-Ray, Datadog

## 6. SPANS

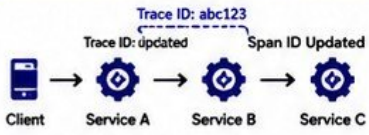



- A span represents a unit of work
- Has start time, end time, duration
- Contains tags, logs, and metadata
- Spans form a trace tree

**SPAN EXAMPLE**

| | |
|---|---|
| Span Name: | GET /api/orders |
| Service: | order-service |
| Start Time: | 12:30:00.123 |
| Duration: | 120ms |
| Tags: | method=GET, status=200 |
| Logs: | db.query=SELECT ... |

## 7. TRACE CONTEXT




- Trace context flows with the request
- Contains Trace ID and Span ID
- Propagated via headers (e.g., W3C Trace Context)
- Maintains continuity across services

**W3C TRACE CONTEXT HEADERS**

`traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01`

`tracestate: vendor=value`

## 8. OPENTELEMETRY



- Open standard for observability
- Collects logs, metrics, and traces
- Vendor-neutral instrumentation
- Rich ecosystem and integrations
- One pipeline, multiple backends

**OTEL COMPONENTS**

- Instrumentation — Add telemetry to code
- SDK — Process and export data
- Exporter — Send to backend

## 9. ROOT CAUSE INVESTIGATION

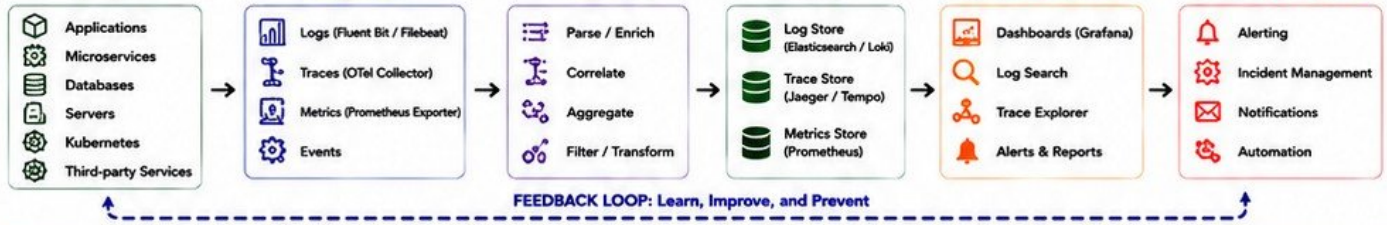


- Use logs + metrics + traces
- Follow the request path
- Identify failing component
- Analyze context and timing
- Fix the root cause, not just the symptom

**INVESTIGATION WORKFLOW**

1. Detect Issue (Alert / User Report)
2. Check Metrics (What is happening?)
3. Check Logs (What errors occurred?)
4. Follow Trace (Where did it fail?)
5. Analyze & Identify Root Cause
6. Fix, Verify, and Monitor

## OBSERVABILITY ARCHITECTURE




## BEST PRACTICES

- Use structured logs consistently
- Always include correlation IDs
- Keep timestamps in UTC
- Retain logs and traces appropriately
- Monitor the monitoring system
- Continuously improve observability


VERIQTA SOCIAL

YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

VERIQTA
ENGINEERING EXCELLENCE



SRE

# 27. SRE FUNDAMENTALS

**BUILD RELIABLE SYSTEMS. DELIVER VALUE. REDUCE TOIL.**

## 1 RELIABILITY ENGINEERING

- Apply software engineering to operations problems
- Build systems that are scalable, resilient, and easy to operate
- Measure, learn, and continuously improve
- Balance reliability, speed, and cost

## 2 SLIs (SERVICE LEVEL INDICATORS)

- Quantify what matters to users
- Examples: Availability, Latency, Error Rate, Throughput, Success Rate
- Good SLIs are actionable, measurable, and user-centric

## 3 SLOs (SERVICE LEVEL OBJECTIVES)

- Set objective targets for your SLIs
- Define the level of reliability you commit to
- Example: 99.9% availability over 30 days
- SLOs guide engineering and product decisions

## 4 SLAs (SERVICE LEVEL AGREEMENTS)

- Formal contract with customers
- Typically based on SLOs
- Includes commitments and consequences when unmet
- Legal and business implications

## 5 ERROR BUDGETS

- The acceptable amount of unreliability
- Error Budget = (1 - SLO) over time
- When budget is exhausted, stop risky changes
- When budget is healthy, you can innovate faster

## 6 AVAILABILITY

99.9%

- System is operational and accessible
- Formula: Uptime / (Uptime + Downtime)
- Common targets: 99%, 99.9%, 99.99%, 99.999%
- Design for high availability and resilience

## 7 LATENCY

- How fast your system responds
- Measured at multiple percentiles (P50, P95, P99, P99.9)
- Tail latency impacts user experience
- Monitor and optimize critical paths

## 8 RELIABILITY TARGETS

- Define the reliability bar for each service
- Align with business needs and user impact
- Reassess as systems evolve
- Use data to validate and adjust targets

## 9 TOIL

- Manual, repetitive, operational work
- Does not scale
- Reduces engineer satisfaction
- Automate, simplify, and eliminate toil
- Measure toil and track reduction

## 10 RELIABILITY VS DELIVERY SPEED

- Finding the right balance is key
- Use error budgets to make smart trade-offs
- Reliability enables sustainable delivery
- Fast delivery without reliability is risky

## HOW SRE PRINCIPLES WORK TOGETHER

DEFINE USER EXPERIENCE → MEASURE WITH SLIs → SET SLOs → MANAGE ERROR BUDGETS → REDUCE TOIL AND IMPROVE → RELIABLE SYSTEMS HAPPY USERS

## EXAMPLE: AVAILABILITY SLO

SLO: 99.9% availability over 30 days
Allowed Downtime:
0.1% of 30 days = 43.2 minutes
Error Budget: 43.2 minutes
If exhausted, pause non-essential changes.

## SRE GOLDEN SIGNALS

| | |
|---|---|
| Latency | Traffic |
| Errors | Saturation |

## KEY TAKEAWAYS

- Start with what matters to users (SLIs).
- Set realistic SLOs and communicate them.
- Use error budgets to drive decisions.
- Reduce toil to improve reliability.
- Balance speed and reliability.
- Reliability is an engineering feature.

**REMEMBER:** SRE IS NOT JUST ABOUT TOOLS. IT IS A MINDSET OF RELIABILITY, OWNERSHIP, AND CONTINUOUS IMPROVEMENT.

VERIQTA
ENGINEERING EXCELLENCE

DEVOPS BIBLE
40-PAGE HANDBOOK

# 28. DEVSECOPS FUNDAMENTALS

DEV SEC OPS

**BUILD SECURE. DELIVER FAST. PROTECT CONTINUOUSLY.**

## INTEGRATE SECURITY INTO EVERY STAGE OF THE DELIVERY LIFECYCLE

### 1 SHIFT-LEFT SECURITY

- Move security activities earlier in the lifecycle
- Find and fix issues before they reach production
- Reduce cost and risk
- Build security into culture and process

### 2 SECURE SDLC

- Security in every SDLC phase
- Plan, Code, Build, Test, Release, Deploy, Operate
- Threat modeling
- Risk assessment
- Continuous feedback

### 3 SAST (STATIC APPLICATION SECURITY TESTING)

- Analyze source code without executing
- Detect vulnerabilities early
- Integrate in CI pipeline
- Tools: SonarQube, Semgrep, Checkmarx

### 4 DAST (DYNAMIC APPLICATION SECURITY TESTING)

- Test running application from the outside
- Find runtime vulnerabilities
- Simulate real attacks
- Tools: OWASP ZAP, Burp Suite, Invicti

### 5 SCA (SOFTWARE COMPOSITION ANALYSIS)

- Analyze open source dependencies
- Identify known vulnerabilities
- Track license compliance
- Tools: Snyk, OWASP Dependency-Check

### 6 SECRET SCANNING

- Detect hardcoded secrets and credentials
- Prevent secret leaks in code
- Scan commits and repos
- Tools: GitGuardian, Gitleaks, TruffleHog

### 7 CONTAINER SCANNING

- Scan container images for vulnerabilities
- Check OS packages and configurations
- Use minimal and trusted base images
- Tools: Trivy, Clair, Anchore

### 8 INFRASTRUCTURE SCANNING

- Scan IaC, cloud resources, and configurations
- Detect misconfigurations and exposed resources
- Ensure compliance
- Tools: Checkov, tfsec, AWS Inspector, ScoutSuite

### 9 POLICY ENFORCEMENT

- Enforce security and compliance policies
- Use OPA, Sentinel, Kyverno, or Conftest
- Prevent drift and insecure changes
- Apply guardrails everywhere

### 10 SECURITY GATES

- Stop bad code, configs, or images from progressing
- Gate on quality, security, and compliance
- Fail fast, fix early
- Protect production continuously

## DEVSECOPS LIFECYCLE WITH SECURITY INTEGRATION

| PLAN | CODE | BUILD | TEST | RELEASE | DEPLOY | OPERATE |
|---|---|---|---|---|---|---|
| Threat Modeling<br>Risk Assessment | SAST<br>Secret Scanning<br>SCA | Dependency Check<br>Container Scan<br>IaC Scan | DAST<br>API Security<br>Unit / Integration Tests | Security Gates<br>Policy Checks<br>Approval | Infrastructure Scan<br>Config Validation<br>Image Verification | Monitoring<br>Vulnerability Mgmt<br>Incident Response |

## KEY PRINCIPLES

- SECURITY IS EVERYONE'S RESPONSIBILITY
- AUTOMATE SECURITY CHECKS
- FAIL FAST AND FIX EARLY
- CONTINUOUS VISIBILITY AND FEEDBACK
- PROTECT THE BUSINESS, ENABLE INNOVATION

## BENEFITS

- REDUCE RISK AND VULNERABILITIES
- COMPLY WITH STANDARDS AND REGULATIONS
- IMPROVE CODE QUALITY AND RELIABILITY
- FASTER DELIVERY WITH CONFIDENCE
- BUILD TRUST WITH USERS AND STAKEHOLDERS

**REMEMBER:** SECURITY IS NOT A CHECKLIST. IT IS A CULTURE, A PROCESS, AND A CONTINUOUS PRACTICE.

VERIQTA
ENGINEERING EXCELLENCE



# 29. IDENTITY, SECRETS, AND ACCESS CONTROL

**CONTROL WHO CAN ACCESS WHAT. PROTECT EVERY IDENTITY AND SECRET.**

## SECURE ACCESS. PROTECT SECRETS. REDUCE RISK.

### 1 AUTHENTICATION

- Verify who the user or system is
- Methods: Passwords, MFA, Certificates, Tokens
- Strong authentication prevents unauthorized access
- Enforce MFA everywhere

### 2 AUTHORIZATION

- Determine what an authenticated identity can do
- Based on roles, policies, and permissions
- Principle of least privilege
- Continuously review access

### 3 IAM (IDENTITY & ACCESS MANAGEMENT)

- Manage users, groups, roles, and policies
- Centralize identity and access control
- Federated identity and SSO
- Enable audit and compliance

### 4 RBAC (ROLE-BASED ACCESS CONTROL)

- Grant permissions based on roles
- Roles map to permissions
- Scalable and easy to manage
- Widely used in Kubernetes and cloud platforms

### 5 LEAST PRIVILEGE

- Grant only the minimum access required
- Reduces blast radius
- Prevents accidental or malicious actions
- Review and remove unnecessary access

### 6 SERVICE ACCOUNTS

- Non-human identities for applications and services
- Scoped permissions only
- Use per service / workload
- Avoid using human credentials

### 7 SECRETS MANAGEMENT

- Store secrets securely
- Encrypt in transit and at rest
- Centralized secret stores (Vault, AWS Secrets Manager, Azure Key Vault)
- Control access strictly

### 8 SECRET ROTATION

- Rotate secrets regularly
- Reduce exposure window
- Automate rotation whenever possible
- Revoke old secrets immediately

### 9 SHORT-LIVED CREDENTIALS

- Use temporary credentials with short TTL
- Reduce risk of credential leakage
- Prefer tokens, STS, or OIDC
- Avoid long-lived keys

### 10 WORKLOAD IDENTITY

- Bind workloads to identities (OIDC, IRSA, Workload Identity)
- No static credentials inside workloads
- Cloud-native and more secure
- Enables fine-grained access control

## HOW SECURE IDENTITY AND ACCESS WORK TOGETHER

**AUTHENTICATE** — Verify identity → **AUTHORIZE** — Check permissions → **APPLY LEAST PRIVILEGE** — Limit access → **MANAGE SECRETS** — Store securely → **ROTATE SECRETS REGULARLY** — Reduce risk → **USE SHORT-LIVED CREDENTIALS** — Minimize exposure → **WORKLOAD IDENTITY** — Secure workloads

## BEST PRACTICES

- Enforce MFA for all users
- Use RBAC and least privilege everywhere
- Store secrets in a dedicated secret manager
- Rotate secrets and keys regularly
- Use short-lived credentials and workload identity
- Audit access and permissions continuously

## KEY TAKEAWAY

**IDENTITIES CAN BE COMPROMISED. PERMISSIONS SHOULD NOT BE. SECURE ACCESS IS CONTINUOUS WORK.**

## COMMON RISKS

- Overly permissive access
- Sharing credentials
- Hardcoded secrets
- Long-lived credentials
- No secret rotation
- Missing audit and reviews

**REMEMBER:** THE RIGHT IDENTITY. THE RIGHT ACCESS. ONLY WHEN NEEDED. FOR ONLY AS LONG AS NEEDED.

VERIQTA
ENGINEERING EXCELLENCE



# 30. SOFTWARE SUPPLY CHAIN SECURITY

**SECURE EVERY COMPONENT. TRUST EVERY STEP. PROTECT THE CHAIN.**

## A SECURE SUPPLY CHAIN BUILDS TRUST AND PREVENTS COMPROMISE.

### 1 DEPENDENCY RISKS

- Dependencies can contain vulnerabilities or backdoors
- Transitive dependencies increase risk
- Compromised packages can impact your systems
- Continuously assess and reduce risk

### 2 ARTIFACT INTEGRITY

- Ensure artifacts are not tampered with
- Verify checksums and hashes
- Detect unauthorized changes early
- Protect the integrity of binaries and images

### 3 SBOM (SOFTWARE BILL OF MATERIALS)

- Maintain a complete list of components
- Understand licenses and vulnerabilities
- Improve transparency and compliance
- Essential for risk management

### 4 SIGNED ARTIFACTS

- Sign artifacts with trusted keys
- Verify signatures before use or deployment
- Prevent tampering and impersonation
- Build trust in the delivery pipeline

### 5 PROVENANCE

- Track where artifacts come from
- Trace the full build and delivery path
- Know who built, tested, and signed
- Critical for forensics and compliance

### 6 DEPENDENCY PINNING

- Lock dependencies to specific versions
- Prevent unexpected updates or breaks
- Ensure reproducible builds
- Reduce risk from malicious or vulnerable updates

### 7 TRUSTED REGISTRIES

- Use trusted and verified package registries
- Avoid unofficial or public unaudited sources
- Mirror and cache internally when possible
- Enforce allow-list of trusted sources

### 8 CI/CD SECURITY

- Secure the entire pipeline
- Protect build agents and runners
- Scan code, dependencies, and images
- Enforce quality and security gates
- Fail fast on violations

### 9 THIRD-PARTY ACTIONS

- Review and validate all third-party actions
- Pin actions to specific commits
- Limit permissions of actions
- Monitor for suspicious behavior

### 10 SUPPLY CHAIN ATTACK PREVENTION

- Use defense in depth
- Apply least privilege everywhere
- Continuously monitor and detect threats
- Keep systems and dependencies up to date
- Prepare, detect, respond, and recover

## SECURE SOFTWARE SUPPLY CHAIN WORKFLOW

**CODE & DEPENDENCIES** — Write code & declare dependencies
→ **ASSESS & SCAN** — Scan for vulnerabilities and license issues
→ **BUILD & TEST** — Build, test, and validate security
→ **SIGN & VERIFY** — Sign artifacts and verify integrity
→ **PUBLISH** — Publish to trusted registries
→ **DEPLOY** — Deploy with verified and trusted artifacts
→ **MONITOR & RESPOND** — Continuously monitor and respond to threats

## KEY PRINCIPLES

- TRUST NOTHING. VERIFY EVERYTHING.
- MINIMIZE AND CONTROL DEPENDENCIES.
- AUTOMATE SECURITY IN THE PIPELINE.
- USE SIGNED AND VERIFIABLE ARTIFACTS.
- CONTINUOUSLY MONITOR AND IMPROVE.

## WHAT CAN GO WRONG?

- MALICIOUS PACKAGES
- COMPROMISED BUILDS
- TAMPERED ARTIFACTS
- UNTRUSTED SOURCES
- SUPPLY CHAIN ATTACKS

## BEST PRACTICES

- MAINTAIN SBOM FOR ALL ARTIFACTS.
- PIN DEPENDENCIES AND ACTIONS.
- USE PRIVATE REGISTRIES WHEN POSSIBLE.
- ENFORCE SECURITY GATES IN CI/CD.
- REVIEW, AUDIT, AND IMPROVE CONTINUOUSLY.

**REMEMBER:** A STRONG SUPPLY CHAIN IS A SHARED RESPONSIBILITY ACROSS PEOPLE, PROCESS, AND TECHNOLOGY.

VERIQTA
ENGINEERING EXCELLENCE



# 31. INCIDENT RESPONSE

**DETECT FAST. RESPOND SMART. RESTORE RELIABLY.**

**A STRUCTURED APPROACH TO RESOLVE INCIDENTS AND MINIMIZE IMPACT**

## 1 DETECTION

- Monitor systems and services
- Detect anomalies early
- Use metrics, logs and alerts
- Identify potential impact
- Confirm incident existence

## 2 ALERTING

- Trigger alerts based on rules
- Reduce noise and false alerts
- Route alerts to the right team
- Include context in alerts
- Ensure alert reliability

## 3 TRIAGE

- Validate the incident
- Identify affected systems
- Determine impact and scope
- Gather initial data
- Decide next actions

## 4 SEVERITY CLASSIFICATION

- Classify severity (P1–P4)
- Consider impact and urgency
- Use defined severity matrix
- Communicate severity
- Reassess as needed

## 5 INCIDENT COMMAND

- Assign incident commander
- Define roles and responsibilities
- Establish decision authority
- Coordinate response efforts
- Keep command structure clear

## 6 COMMUNICATION

- Communicate early and clearly
- Provide regular updates
- Use status pages / channels
- Keep stakeholders informed
- Document all communications

## 7 MITIGATION

- Contain the impact
- Apply temporary fixes
- Reduce user impact
- Avoid further degradation
- Document mitigation steps

## 8 RECOVERY

- Apply permanent fix
- Restore normal operations
- Monitor system behavior
- Validate dependencies
- Ensure stability

## 9 VERIFICATION

- Verify system is healthy
- Confirm issue is resolved
- Validate with monitoring
- Test critical user flows
- Get confirmation from team

## 10 INCIDENT TIMELINE

- Record key timestamps
- Track events in order
- Include actions taken
- Use timeline for analysis
- Support post-incident review

## 11 ESCALATION

- Escalate when needed
- Follow escalation path
- Engage experts / teams
- Remove blockers quickly
- Escalate based on severity

## PRINCIPLES

- Focus on users and impact
- Communicate clearly
- Act quickly and decisively
- Document everything
- Learn and improve
- Blameless culture

## INCIDENT RESPONSE FLOW




**REMEMBER:** PREPARE BEFORE INCIDENTS. PRACTICE RESPONSE. REVIEW AFTERWARDS.
EVERY INCIDENT IS A LESSON. EVERY LESSON IMPROVES RELIABILITY.

# 32. PRODUCTION TROUBLESHOOTING METHODOLOGY

A SYSTEMATIC APPROACH TO FIND, FIX, AND PREVENT ISSUES IN PRODUCTION

| | | |
|---|---|---|
| 1 | DEFINE SYMPTOMS | Identify and document the problem clearly. What is broken? What are users experiencing? |
| 2 | ESTABLISH SCOPE | Determine the impact and scope. Who is affected? Which services or systems? |
| 3 | CHECK RECENT CHANGES | Review deployments, configuration changes, releases, and infrastructure updates. |
| 4 | INSPECT METRICS | Check monitoring dashboards and metrics. Look for anomalies, spikes, or drops. |
| 5 | INSPECT LOGS | Search logs for errors, warnings, and correlated events. |
| 6 | INSPECT TRACES | Follow requests across services. Identify slow calls and failures. |
| 7 | TEST DEPENDENCIES | Verify health of downstream and upstream services, databases, and external APIs. |
| 8 | FORM HYPOTHESES | Based on gathered data, create possible root cause theories. |
| 9 | VALIDATE HYPOTHESES | Test and confirm (or eliminate) hypotheses with evidence. |
| 10 | MITIGATE SAFELY | Implement the safest fix or workaround possible. Minimize risk and blast radius. |
| 11 | VERIFY RECOVERY | Confirm the issue is resolved. Monitor to ensure stability. |

## KEY PRINCIPLES

- Stay calm and focused
- Follow the process
- Gather data before acting
- Communicate clearly
- Document everything
- Learn and improve

## TROUBLESHOOTING TOOLS

- Monitoring: Prometheus, Grafana
- Logging: ELK, Loki, CloudWatch
- Tracing: Jaeger, OpenTelemetry
- Alerts: Alertmanager, PagerDuty
- APM: Datadog, New Relic
- Status: Uptime, Health Checks

## WHAT TO AVOID

- Assuming the root cause
- Making random changes
- Ignoring user impact
- Skipping steps
- No communication
- Not documenting

**REMEMBER:** Data → Hypothesis → Validation → Safe Mitigation → Verification

THIS PROCESS SAVES TIME, REDUCES IMPACT, AND BUILDS RELIABILITY.

VERIQTA
ENGINEERING EXCELLENCE



# 33. COMMON PRODUCTION FAILURE PATTERNS

**RECOGNIZE FAST. DIAGNOSE ACCURATELY. RESOLVE SAFELY.**

## 1 CPU SATURATION




**SYMPTOMS**
- High CPU usage
- Slow response
- Request timeouts

**COMMON CAUSES**
- Runaway processes
- Unoptimized code
- Too many requests
- Background jobs

**WHAT TO CHECK**
- top, htop
- CPU throttling
- Application logs
- Recent deployments

## 2 MEMORY EXHAUSTION



**SYMPTOMS**
- High memory usage
- OOM kills
- Service crashes

**COMMON CAUSES**
- Memory leaks
- Large caches
- Too many instances
- Insufficient limits

**WHAT TO CHECK**
- `free -m`, top
- dmesg (OOM logs)
- Application metrics
- Container limits

## 3 DISK FULL



**SYMPTOMS**
- Writes failing
- Services crashing
- Logs not writing

**COMMON CAUSES**
- Large logs
- Unclean temp files
- No log rotation
- Full partitions

**WHAT TO CHECK**
- `df -h`
- Disk I/O (`iostat`)
- Log directories
- Inode usage (`df -i`)

## 4 DNS FAILURE




**SYMPTOMS**
- Cannot resolve names
- Intermittent failures
- Slow requests

**COMMON CAUSES**
- DNS server down
- Wrong DNS config
- TTL / cache issues
- DNS rate limits

**WHAT TO CHECK**
- nslookup / dig
- /etc/resolv.conf
- CoreDNS / DNS logs
- Network policies

## 5 NETWORK TIMEOUT



**SYMPTOMS**
- Request timeouts
- Slow connections
- Intermittent failures

**COMMON CAUSES**
- Network congestion
- Packet loss
- Firewall / security group
- Unstable routes

**WHAT TO CHECK**
- `ping`, traceroute
- Security groups
- Network latency
- Packet loss (`mtr`)

## 6 DATABASE EXHAUSTION



**SYMPTOMS**
- DB connections failing
- Slow queries
- High DB CPU / I/O

**COMMON CAUSES**
- Too many connections
- Long running queries
- No connection limits
- Resource exhaustion

**WHAT TO CHECK**
- DB metrics
- Active connections
- Slow query logs
- Resource limits

## 7 CONNECTION POOL PROBLEMS



**SYMPTOMS**
- Connection timeouts
- Requests hanging
- Pool exhausted errors

**COMMON CAUSES**
- Pool too small
- Connections not released
- Leak in application
- Idle timeout mismatch

**WHAT TO CHECK**
- Pool metrics
- Active / idle connections
- Application logs
- Pool configuration

## 8 CERTIFICATE EXPIRATION



**SYMPTOMS**
- SSL errors
- Services failing
- Users see warnings

**COMMON CAUSES**
- Expired certificate
- Auto-renew failed
- Wrong certificate
- Missing chain

**WHAT TO CHECK**
- openssl s_client
- Expiration date
- Certificate chain
- Monitoring alerts

## 9 DEPENDENCY FAILURE



**SYMPTOMS**
- External calls failing
- Errors from API
- Timeouts

**COMMON CAUSES**
- Service down
- Rate limits
- Network issues
- Bad configuration

**WHAT TO CHECK**
- Status page
- Error logs
- Retries / timeouts
- Circuit breakers

## 10 BAD DEPLOYMENT



**SYMPTOMS**
- Errors after deploy
- Health check fails
- Performance drop

**COMMON CAUSES**
- Bug in new release
- Wrong config
- Database migration issues
- Incomplete rollout

**WHAT TO CHECK**
- Deployment logs
- Health checks
- Application logs
- Recent changes

## 11 CONFIGURATION DRIFT



**SYMPTOMS**
- Unexpected behavior
- Inconsistent environments
- Policy violations

**COMMON CAUSES**
- Manual changes
- No IaC
- Missing automation
- Untracked changes

**WHAT TO CHECK**
- IaC plan / diff management
- Current state vs desired state

## GENERAL TROUBLESHOOTING CHECKLIST

- [x] Stay calm and acknowledge the issue
- [x] Confirm the impact and scope
- [x] Collect data before making changes
- [x] Check recent changes and deployments
- [x] Review metrics, logs, and traces
- [x] Test dependencies and external services
- [x] Form a hypothesis and validate
- [x] Mitigate the issue safely
- [x] Verify recovery and monitor
- [x] Document the incident and root cause

## PREVENTION BEST PRACTICES

- [x] Monitor everything that matters
- [x] Set meaningful alerts
- [x] Automate deployments safely
- [x] Use IaC for consistency
- [x] Keep dependencies healthy
- [x] Run chaos and failure tests
- [x] Review capacity regularly
- [x] Document runbooks
- [x] Learn from every incident
- [x] Continuously improve systems



**REMEMBER:** MOST OUTAGES ARE PREVENTABLE. OBSERVE → UNDERSTAND → ACT → VERIFY → IMPROVE

VERIQTA
ENGINEERING EXCELLENCE



# 34. ROLLBACK, RECOVERY, AND DISASTER RECOVERY

PREPARE. PROTECT. RECOVER. VALIDATE.

## 1 APPLICATION ROLLBACK

Revert application to a previous known good version.

### KEY ACTIONS
- Use previous release
- Revert deployment
- Rollback traffic
- Verify functionality

### WHAT TO CHECK
- Deployment status
- Version running
- Health checks
- User experience

## 2 INFRASTRUCTURE ROLLBACK

Revert infrastructure changes to a stable state.

### KEY ACTIONS
- Revert IaC changes
- Restore previous state
- Roll back resources
- Validate environment

### WHAT TO CHECK
- Resource status
- Configuration drift
- Dependencies
- Service availability

## 3 DATABASE RECOVERY

Restore database to a consistent and healthy state.

### KEY ACTIONS
- Restore from backup
- Point-in-time recovery
- Repair or rebuild
- Validate data integrity

### WHAT TO CHECK
- Data integrity
- Replication status
- Performance
- Application access

## 4 BACKUPS

Maintain reliable backups for all critical systems and data.

### KEY ACTIONS
- Automated backups
- Encrypt backups
- Store offsite / off-region
- Retention policies

### WHAT TO CHECK
- Backup success
- Backup size
- Backup integrity
- Backup age

## 5 RESTORE TESTING

Regularly test restores to ensure backups actually work.

### KEY ACTIONS
- Test full restore
- Test partial restore
- Validate data
- Document results

### WHAT TO CHECK
- Restore success
- Data accuracy
- Recovery time
- Runbook updates

## 6 RPO (RECOVERY POINT OBJECTIVE)

Maximum acceptable data loss measured in time.

### KEY ACTIONS
- Define RPO per system
- Align backup frequency
- Monitor data freshness
- Meet business needs

### WHAT TO CHECK
- Backup intervals
- Replication lag
- Data freshness
- Compliance

## 7 RTO (RECOVERY TIME OBJECTIVE)

Maximum acceptable downtime to restore service.

### KEY ACTIONS
- Define RTO per system
- Optimize recovery time
- Automate recovery
- Test regularly

### WHAT TO CHECK
- Recovery runbooks
- Dependencies
- Automation status
- Recovery metrics

## 8 DISASTER RECOVERY

Recover systems after a major disruption or site failure.

### KEY ACTIONS
- Activate DR plan
- Failover services
- Communicate status
- Stabilize systems

### WHAT TO CHECK
- DR environment
- Critical services
- Data integrity
- Business operations

## 9 MULTI-REGION RECOVERY

Use multiple regions to ensure high availability and recovery.

### KEY ACTIONS
- Deploy across regions
- Replicate data
- Configure failover
- Route traffic

### WHAT TO CHECK
- Region health
- Replication status
- Failover routing
- Latency & performance

## 10 RECOVERY VALIDATION

Validate that systems are fully recovered and operating correctly.

### KEY ACTIONS
- Run validation tests
- Verify functionality
- Monitor stability
- Close incident

### WHAT TO CHECK
- Service health
- User experience
- Performance
- Monitoring alerts

## RECOVERY STRATEGY CHECKLIST

- [x] Identify critical systems and data
- [x] Define RPO and RTO for each system
- [x] Automate backups and store securely
- [x] Test restores regularly
- [x] Document runbooks and procedures
- [x] Implement monitoring and alerting
- [x] Review and update DR plan regularly
- [x] Train the team and run DR drills

## RPO VS RTO

| RPO | VS | RTO |
|---|---|---|
| How much data you can afford to lose. | | How long it takes to restore service and operate. |
| Measured in time. | | Measured in time. |

## KEY PRINCIPLES

- [x] Expect failure. Plan for it.
- [x] Automate everything possible.
- [x] Test before you need it.
- [x] Document clearly.
- [x] Communicate effectively.
- [x] Keep it simple and reliable.
- [x] Validate recovery end-to-end.
- [x] Continuously improve.

**REMEMBER:** BEST DISASTER RECOVERY IS NOT JUST BACKUPS — IT IS TESTED, VALIDATED, AND PROVEN TO WORK WHEN YOU NEED IT MOST.

VERIQTA
ENGINEERING EXCELLENCE



# 35. POSTMORTEMS AND CONTINUOUS IMPROVEMENT

LEARN FROM EVERY INCIDENT. IMPROVE EVERY DAY.

## 1 BLAMELESS POSTMORTEMS

Focus on the system, not individuals.

### KEY POINTS

- No blame
- Open and honest
- Safe environment
- Everyone can speak up

### OUTCOME

Trust, transparency, and real learning.

## 2 INCIDENT TIMELINE

Build a clear timeline of what happened.

### KEY POINTS

- When it started
- What changed
- When it escalated
- When mitigated
- When resolved

### OUTCOME

Shared understanding of the sequence.

## 3 ROOT CAUSE

Identify the underlying cause of the incident.

### KEY POINTS

- Go beyond symptoms
- Find the real trigger
- Ask "Why did this happen?"

### OUTCOME

Address the real problem, not just the symptoms.

## 4 CONTRIBUTING FACTORS

List all factors that made the incident worse.

### KEY POINTS

- Technical factors
- Process gaps
- People factors
- Environmental factors
- Tooling gaps

### OUTCOME

Understand the full context and impact.

## 5 FIVE WHYS

Keep asking "Why?" until you reach the root cause.

### KEY POINTS

- Why did it happen?
- Why did that happen?
- Keep digging deeper
- Stop at root cause

### OUTCOME

Clear, deep, and verified root cause.

## 6 CORRECTIVE ACTIONS

Actions to fix the issue quickly and restore stability.

### KEY POINTS

- Immediate fixes
- Hotfixes
- Rollbacks
- Workarounds
- System recovery

### OUTCOME

Restore service and stabilize the system.

## 7 PREVENTIVE ACTIONS

Actions to prevent this incident from happening again.

### KEY POINTS

- Process improvements
- Automation
- Monitoring & alerts
- Better testing
- Documentation

### OUTCOME

Stronger systems and fewer future incidents.

## 8 OWNERSHIP

Assign clear ownership for each action item.

### KEY POINTS

- One owner per action
- Clear responsibilities
- Empower the owner
- Provide resources
- Set expectations

### OUTCOME

Accountability and successful follow-through.

## 9 FOLLOW-UP

Track progress until all actions are completed.

### KEY POINTS

- Track action status
- Regular updates
- Remove blockers
- Verify completion
- Close the loop

### OUTCOME

Actions completed and risks truly reduced.

## 10 LEARNING FROM INCIDENTS

Capture lessons and share knowledge.

### KEY POINTS

- Share learnings
- Update runbooks
- Improve training
- Improve systems
- Build a learning culture

### OUTCOME

Continuous improvement and stronger reliability.

## EFFECTIVE POSTMORTEM CHECKLIST

- ✓ Create a clear and factual incident timeline
- ✓ Identify the root cause using data and analysis
- ✓ List all contributing factors (not just technical)
- ✓ Use the Five Whys to validate the root cause
- ✓ Define corrective actions taken during the incident
- ✓ Define preventive actions to reduce future risk
- ✓ Assign ownership for every action item
- ✓ Set target dates and track progress
- ✓ Verify completion and measure effectiveness
- ✓ Share learnings across the organization

## POSTMORTEM PRINCIPLES

- Be blameless
- Be honest and transparent
- Focus on systems, not people
- Learn and improve
- Drive meaningful change
- Build a culture of reliability

**REMEMBER:** EVERY INCIDENT IS AN OPPORTUNITY TO IMPROVE.
CAPTURE LESSONS. TAKE ACTION. PREVENT RECURRENCE.

VERIQTA
ENGINEERING EXCELLENCE



# 36. PLATFORM ENGINEERING AND INTERNAL DEVELOPER PLATFORMS

BUILDING SELF-SERVICE PLATFORMS THAT EMPOWER DEVELOPERS AND ACCELERATE DELIVERY

## 1 PLATFORM ENGINEERING

- Build and operate platforms for internal users
- Abstract complexity and standardize best practices
- Enable developers to ship faster and safer
- Focus on scalability, reliability, security, and automation
- Treat the platform like a product

## 2 DEVELOPER EXPERIENCE

- Frictionless workflows
- Fast onboarding
- Clear documentation
- Self-service tooling
- Consistent environments
- Quick feedback loops
- Happy developers ship better software

## 3 INTERNAL DEVELOPER PLATFORMS

- Central portal for all platform services
- Unified tooling and services
- Discoverable and easy to use
- Self-service infrastructure, pipelines, and components
- Reduces cognitive load for developers

## 4 GOLDEN PATHS

- Opinionated, secure, and well-architected paths
- Pre-built patterns for common use cases
- Reduce decision fatigue
- Promote consistency and best practices
- Pave the way for fast and reliable delivery

## 5 SELF-SERVICE INFRASTRUCTURE

- Provision environments on demand
- Automated and API-driven
- No manual tickets
- Policy-driven and secure
- Scalable and cost-effective
- Infrastructure as a product

## 6 SERVICE TEMPLATES

- Pre-approved service blueprints
- Include code, IaC, pipelines, testing, and monitoring
- Consistent, secure, and production-ready
- Speeds up new service delivery
- Easier maintenance and upgrades

## 7 DEVELOPER PORTALS

- Single place to discover and access everything
- Catalogs for services, templates, and tools
- Guides, documentation, and runbooks
- Requests, approvals, and tracking
- Great DX starts with a great portal

## 8 PLATFORM APIs

API

- Expose platform capabilities via well-documented APIs
- Automate and integrate everything
- Enable self-service and extensibility
- Secure, versioned, and reliable APIs
- APIs are the backbone of your platform

## 9 GUARDRAILS

- Enforce policies and compliance
- Security, cost, and performance controls
- Prevent misconfigurations
- Automated policy checks
- Freedom to innovate within safe boundaries

## 10 PLATFORM AS A PRODUCT

- Platform has users (customers)
- Measure usage and value
- Gather feedback
- Iterate and improve
- Clear SLAs and reliability standards
- Ownership, roadmap, and support

## INTERNAL DEVELOPER PLATFORM ARCHITECTURE

**DEVELOPERS**
- Build
- Test
- Deploy
- Operate

→ **DEVELOPER PORTAL**
- Catalogs
- Documentation
- Templates
- Self-Service

→ **PLATFORM SERVICES**
- CI/CD Pipelines
- IaC Provisioning
- Environments Management
- Observability & Monitoring
- Security & Compliance
- Service Templates
- APIs & Integrations
- Cost Management

→ **RUN ANYWHERE**
- Cloud
- On-Prem
- Hybrid

AUTOMATION • STANDARDIZATION • SELF-SERVICE • OBSERVABILITY • SECURITY

## PLATFORM ENGINEERING BENEFITS

- FASTER DELIVERY
- HIGHER QUALITY
- IMPROVED DEVELOPER EXPERIENCE
- STANDARDIZATION AT SCALE
- REDUCED MANUAL WORK
- BETTER SECURITY AND COMPLIANCE
- LOWER COSTS
- SCALABLE AND RELIABLE PLATFORMS
- MORE TIME FOR INNOVATION

**A GREAT PLATFORM EMPOWERS DEVELOPERS. A STRONG PLATFORM SCALE THE BUSINESS.**

VERIQTA
ENGINEERING EXCELLENCE



# 37. ENTERPRISE DEVOPS ARCHITECTURE

END-TO-END DEVOPS PLATFORM FOR SCALABLE, SECURE, AND RELIABLE DELIVERY

## END-TO-END ARCHITECTURE OVERVIEW

**1 SOURCE CONTROL**
Git Repositories
Branching Strategy
Pull Requests
Code Reviews

**2 CI PLATFORM**
Build
Test
Scan
Quality Gates
Pipeline as Code

**3 ARTIFACT MANAGEMENT**
JFrog
Store Artifacts
Container Registry
Versioning
Retention

**4 SECURITY SYSTEMS**
SAST / DAST
Container Scanning
Dependency Scanning
Policy Enforcement
Vulnerability Mgmt

**5 INFRASTRUCTURE AS CODE**
Terraform / OpenTofu
Provisioning
Environment Mgmt
State Management
Policy as Code

**USERS**
Developers
Operators
SREs
Business Users

**6 KUBERNETES**
Container Orchestration
Workloads
Scaling
Networking
Storage

**7 GITOPS**
argo
Git as Source of Truth
Declarative Deployments
Drift Detection
Auto Sync
Rollback

**8 OBSERVABILITY**
Prometheus
Grafana
Loki
Metrics
Logs
Traces
Dashboards
Alerting

**9 SECRETS MANAGEMENT**
HashiCorp Vault
Secrets Storage
Dynamic Secrets
Rotation
Access Control
Audit Logs

**10 CLOUD INFRASTRUCTURE**
aws
Compute
Networking
Storage
Databases
Managed Services

**APRLICATIONS**
Reliable
Scalable
Secure
Observable
Available

## ARCHITECTURE FLOW

CODE COMMIT → BUILD & TEST AUTOMATION → ARTIFACT PUBLISHED → SECURITY VALIDATION → INFRA PROVISIONED → DEPLOYED ON KUBERNETES → GITOPS SYNC → MONITOR & OBSERVE → MANAGE SECRETS → RUN IN CLOUD

### 1 SOURCE CONTROL
- Centralized Git repositories
- Branch protection
- Code reviews
- Audit trails
- Collaboration
- Pull requests

### 2 CI PLATFORM
- Automated builds
- Automated tests
- Code quality checks
- Security scanning
- Pipeline as Code
- Fail fast feedback

### 3 ARTIFACT MANAGEMENT
- Store binaries
- Store packages
- Store container images
- Version control
- Retention policies
- Immutability

### 4 SECURITY SYSTEMS
- Shift-left security
- Continuous scanning
- Vulnerability management
- Policy enforcement
- Compliance checks
- Risk visibility

### 5 INFRASTRUCTURE AS CODE
- Versioned IaC
- Reusable modules
- Environment parity
- Automated provisioning
- Drift detection
- Policy as Code

### 6 KUBERNETES
- Container orchestration
- Self-healing
- Auto scaling
- Service discovery
- Networking
- Storage
- High availability

### 7 GITOPS
- Declarative state
- Git as source of truth
- Automated sync
- Drift detection
- Rollback by commit
- Traceable changes

### 8 OBSERVABILITY
- Metrics collection
- Log aggregation
- Distributed tracing
- Dashboards
- Alerting
- SLO monitoring

### 9 SECRETS MANAGEMENT
- Centralized secrets
- Dynamic secrets
- Secrets rotation
- Fine-grained access
- Audit and logging
- Reduce exposure

### 10 CLOUD INFRASTRUCTURE
- Scalable resources
- High availability
- Managed services
- Cost optimization
- Security groups
- Network isolation

### KEY BENEFITS
- FASTER DELIVERY
- HIGHER RELIABILITY
- BETTER SECURITY
- IMPROVED COLLABORATION
- AUTOMATION AT EVERY LAYER
- OBSERVABILITY BY DEFAULT
- SCALABLE AND RESILIENT SYSTEMS

**A WELL-DESIGNED DEVOPS ARCHITECTURE DELIVERS VALUE FASTER, SAFER, AND MORE RELIABLY.**

# 38. DEVOPS PERFORMANCE AND ENGINEERING METRICS

MEASURE WHAT MATTERS. IMPROVE CONTINUOUSLY. DELIVER VALUE FASTER.

## 1 DEPLOYMENT FREQUENCY

How often code is successfully deployed to production.

**WHY IT MATTERS:**
Higher frequency means faster delivery and more feedback.

**GOOD TARGET:**
On-demand or multiple times per day

## 2 LEAD TIME FOR CHANGES

Time from code commit to successful deployment in production.

**WHY IT MATTERS:**
Shorter lead time means faster value delivery to users.

**GOOD TARGET:**
Less than 1 day

## 3 CHANGE FAILURE RATE

Percentage of changes that result in degraded service or require rollback.

**WHY IT MATTERS:**
Lower failure rate means higher quality and stability.

**GOOD TARGET:**
Less than 15%

## 4 MEAN TIME TO RECOVERY

Average time to restore service after an incident or failure.

**WHY IT MATTERS:**
Faster recovery reduces impact on users and the business.

**GOOD TARGET:**
Less than 1 hour

## 5 AVAILABILITY

Percentage of time the system is available and functional.

**WHY IT MATTERS:**
Measures reliability and user trust.

**GOOD TARGET:**
99.9% or higher

## 6 PIPELINE DURATION

Total time taken for a CI/CD pipeline run from start to finish.

**WHY IT MATTERS:**
Shorter pipelines enable faster feedback and quicker releases.

**GOOD TARGET:**
As fast as possible (optimize continuously)

## 7 BUILD SUCCESS RATE

Percentage of builds that complete successfully without failures.

**WHY IT MATTERS:**
High success rate ensures developer productivity and trust.

**GOOD TARGET:**
Greater than 95%

## 8 INFRASTRUCTURE RELIABILITY

How reliable, resilient, and performant your infrastructure is.

**WHY IT MATTERS:**
Reliable infrastructure prevents outages and supports scale.

**GOOD TARGET:**
High uptime, low errors

## 9 ALERT QUALITY

Percentage of alerts that are actionable, accurate, and useful.

**WHY IT MATTERS:**
Good alerts reduce noise and help teams focus on real issues.

**GOOD TARGET:**
High signal, low noise

## 10 ENGINEERING EFFECTIVENESS

How effectively the engineering team delivers value to customers.

**WHY IT MATTERS:**
Measures team impact, productivity, and business outcomes.

**GOOD TARGET:**
Continuously improving

### DORA CORE FOUR METRICS

- Deployment Frequency
- Lead Time for Changes
- Change Failure Rate
- Mean Time to Recovery

THESE FOUR METRICS ARE PROVEN TO CORRELATE STRONGLY WITH ELITE PERFORMING TEAMS.

### METRIC MATURITY LEVELS

| | |
|---|---|
| ELITE | High performance across all metrics |
| HIGH | Consistently better than industry average |
| MEDIUM | Improving but inconsistent |
| LOW | Frequent issues and manual processes |

### BEST PRACTICES

- Measure continuously and automatically
- Use real data, not opinions
- Review metrics regularly as a team
- Focus on improvement, not perfection
- Align metrics with business outcomes
- Remove blockers and improve processes
- Celebrate improvements and learn

### METRIC FORMULA EXAMPLES

**CHANGE FAILURE RATE**

$$\frac{\text{Failed Changes}}{\text{Total Changes}} \times 100$$

**AVAILABILITY**

$$\frac{\text{Uptime}}{\text{Uptime + Downtime}} \times 100$$

**BUILD SUCCESS RATE**

$$\frac{\text{Successful Builds}}{\text{Total Builds}} \times 100$$

**MTTR**

$$\frac{\text{Total Recovery Time}}{\text{Number of Incidents}}$$

YOU CAN'T IMPROVE WHAT YOU DON'T MEASURE. GREAT ENGINEERING IS DRIVEN BY DATA.

VERIQTA
ENGINEERING EXCELLENCE



# 39. SENIOR DEVOPS ENGINEER PRODUCTION PRINCIPLES

TIMELESS PRINCIPLES FOR BUILDING RELIABLE, SCALABLE, AND SECURE SYSTEMS

## 1 AUTOMATE REPETITIVE WORK



Automation increases speed, consistency, and reliability.

**FOCUS ON:**
Automation, Scripting, CI/CD, Self-service

## 2 DESIGN FOR FAILURE



Failures will happen. Design systems that expect and survive them.

**FOCUS ON:**
Resilience, Redundancy, Timeouts, Retries

## 3 PREFER SIMPLE SYSTEMS



Simplicity improves reliability, maintainability, and understanding.

**FOCUS ON:**
KISS, Reduce Complexity, YAGNI

## 4 MAKE CHANGES REVERSIBLE



Always design changes so you can go back quickly and safely.

**FOCUS ON:**
Rollbacks, Feature Flags, Blue/Green, Canary

## 5 REDUCE BLAST RADIUS



Limit the impact of failures to small, isolated areas.

**FOCUS ON:**
Isolation, Segmentation, Least Privilege

## 6 BUILD OBSERVABILITY FIRST



You can't fix what you can't see. Instrument everything.

**FOCUS ON:**
Metrics, Logs, Traces, Alerts, Dashboards

## 7 PROTECT PRODUCTION



Production is the customer experience. Treat it with respect.

**FOCUS ON:**
Access Control, Change Management, Safeguards

## 8 DOCUMENT CRITICAL DECISIONS



Good documentation preserves knowledge and prevents repeat mistakes.

**FOCUS ON:**
Architecture Decisions, Runbooks, Standards

## 9 UNDERSTAND DEPENDENCIES



Know how systems, services, and teams depend on each other.

**FOCUS ON:**
Dependency Maps, Communication, Contracts

## 10 MEASURE BEFORE OPTIMIZING



Measure first. Then optimize with data, not assumptions.

**FOCUS ON:**
Metrics, Baselines, Data-driven Decisions

## 11 TREAT SECURITY AS ENGINEERING



Security is not a step, it's a responsibility in every decision.

**FOCUS ON:**
Threat Modeling, Scanning, Least Privilege, Compliance

## 12 OWN RELIABILITY



You own the outcome. Be accountable for reliability and uptime.

**FOCUS ON:**
SLOs, Error Budgets, Incident Ownership

**SENIOR ENGINEERS BUILD SYSTEMS. PRINCIPLED ENGINEERS BUILD LEGACIES.**

### THE MINDSET THAT SETS SENIOR ENGINEERS APART

- Think Long Term
- Automate Relentlessly
- Design Resilient Systems
- Deliver Value
- Improve Continuously
- Lead by Example

**PRINCIPLES TODAY. RELIABILITY TOMORROW.**

VERIQTA
ENGINEERING EXCELLENCE



# 40. THE COMPLETE DEVOPS PRODUCTION BLUEPRINT

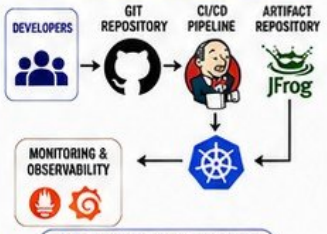
FROM CODE TO PRODUCTION. SECURE. RELIABLE. OBSERVABLE. AUTOMATED.

## END-TO-END DEVOPS PRODUCTION ARCHITECTURE

| 1 CODE | 2 GIT WORKFLOW | 3 CI PIPELINE | 4 SECURITY GATES | 5 ARTIFACT LIFECYCLE | 6 INFRASTRUCTURE PROVISIONING | 7 CONTAINER DEPLOYMENT | 8 KUBERNETES OPERATIONS | 9 GITOPS DELIVERY | 10 MONITORING & OBSERVABILITY | 11 INCIDENT RESPONSE | 12 ROLLBACK & RECOVERY | 13 PRODUCTION READY |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Write Code<br>Commit<br>Changes | Branch<br>Pull Request<br>Review & Merge | Build<br>Test<br>Quality Check | SAST, DAST<br>Scan<br>Policy Check | Store<br>Version<br>Promote | IaC (Terraform)<br>Plan<br>Apply | Build Image<br>Push Image<br>Scan Image | Deploy<br>Run<br>Scale<br>Heal | Git as Source<br>Sync<br>Drift Detect | Metrics, Logs<br>Traces, Alerts<br>Dashboards | Detect<br>Triage<br>Mitigate<br>Communicate | Rollback<br>Restore<br>Recover<br>Verify | Secure<br>Reliable<br>Scalable |

CONTINUOUS FEEDBACK LOOP

### 1. CODE TO PRODUCTION ARCHITECTURE




ALL COMPONENTS CONNECTED.
ALL SYSTEMS OBSERVABLE.
ALL CHANGES TRACEABLE.

### 2. GIT WORKFLOW

- Feature Branching
- Pull Request
- Code Review
- Automated Checks
- Merge to Main
- Protected Branches
- Tags & Releases
- Audit Trail

### 3. CI PIPELINE

- Source Checkout
- Dependency Install
- Build
- Unit Tests
- Integration Tests
- Code Quality
- Security Scan
- Artifact Publish
- Pipeline Feedback

### 4. SECURITY GATES

- SAST (Static Analysis)
- DAST (Dynamic Analysis)
- Dependency Scan
- Container Scan
- Secret Scan
- Policy as Code
- Compliance Check
- Fail on Critical Issues

### 5. ARTIFACT LEFCYCLE

- Versioned Artifacts
- Immutable Artifacts
- Store in Repository
- Promote Across Env
- Retention Policy
- Provenance & SBOM
- Signature & Integrity
- Traceability

JFrog

### 6. INFRASTRUCTURE PROVISIONING

- IaC with Terraform
- Version Controlled
- Reusable Modules
- Plan & Review
- Apply & Automate
- State Management
- Drift Detection
- Environment Isolation

### 7. CONTAINER DEPLOYMENT

- Build Optimized Images
- Scan for Vulnerabilities
- Tag with Version
- Push to Registry
- Immutable Images
- SBOM Generation
- Signature Verification

### 8. KUBERNETES OPERATIONS

- Deploy Applications
- Health Checks
- Auto Scaling
- Resource Limits
- Configuration Mgmt
- Network Policies
- RBAC & Security
- Self-Healing Systems

### 9. GITOPS DELIVERY

- Git is Source of Truth
- Declarative Manifests
- Argo CD Sync
- Drift Detection
- Automated Sync
- Progressive Delivery
- Rollback via Git
- Audit & Traceability

argo

### 10. MONITORING & OBSERVABILITY

- Metrics (Prometheus)
- Logs (Loki / ELK)
- Traces (Jaeger)
- Dashboards (Grafana)
- Alerting
- SLOs & SLIs
- Error Budgets
- Business Metrics

### 11. INCIDENT RESPONSE

- Alert Detection
- Incident Triage
- Severity Assessment
- Communication
- Mitigation
- Escalation
- Postmortem
- Continuous Improvement

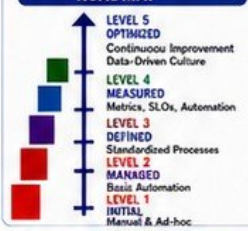
### 12. ROLLBACK & RECOVERY

- Quick Rollback
- Config Rollback
- DB Recovery
- Backup & Restore
- Disaster Recovery
- RTO & RPO
- Recovery Testing
- Verify & Close

### 13. PRODUCTION READINESS CHECKLIST

- Code Reviewed
- Tests Passing
- Security Cleared
- IaC Reviewed
- Observability Ready
- Runbooks Ready
- Backup Validated
- Stakeholder Approval

### 14. DEVOPS MATURITY ROADMAP




### 15. FINAL SENIOR ENGINEER CHECKLIST

- Think in Systems
- Automate Ruthlessly
- Measure Everything
- Prioritize Reliability
- Reduce Complexity
- Secure by Default
- Document Decisions
- Mentor & Improve Team
- Take Ownership
- Deliver Business Value

**GREAT DEVOPS TEAMS DON'T JUST DEPLOY SOFTWARE. THEY DELIVER RELIABILITY, SECURITY, AND VALUE.**